INDEX

S. No.	DATE	TITLE	Page No.
1	26.07.2025	Create a HTML page to display your Bio-Data	
2	26.07.2025	Create a web page to display the details of any two items	
3	28.07.2025	Implement all text and image formatting tags	
4	28.07.2025	Design any web page using Frames	
5	29.07.2025	Implement HTML tables and Lists	
6	29.07.2025	Illustrate Cascading Style Sheet (inline, internal, external and multiple CSS)	
7	30.07.2025	Design any user registration form using all HTML input elements along with CSS	
8	30.07.2025	Display the Emoji symbols using HTML	
9	30.07.2025	Create menus and illustrate drawing borders, shading and shapes in HTML	
10	02.08.2025	Implement all web APIs (drag and drop, Geolocation, Third Party, JavaScript Validation, Web History, Storage, Web Worker)	
11	03.08.2025	Design a User Registration Form in HTML and write a function to validate all the input fields using JavaScript	
12	03.08.2025	Create a small web page using HTML, CSS, JavaScript, PHP and MYSQL.	
13	03.08.2025	Create XML file using your own data	
14	03.08.2025	XML internal DTD	
15	04.08.2025	XML external DTD	
16	04.08.2025	Display XML table in HTML document	
17	04.08.2025	Program to send HTML form data to XML file (use XML and JavaScript)	
18	04.08.2025	Display the data from XML file to HTML forms	
19	04.08.2025	Implement XML scheme (XSD) for simple data type using various restrictions and attributes	
20	04.08.2025	Implement XML scheme (XSD) for complex data type	
21	06.08.2025	Write a program to read XML data using PHP simplexml_load_string() function	
22	06.08.2025	Write a program to read data stored in XML file using PHP	
23	29.08.2025	Write a program to display the XML data using PHP	
24	29.08.2025	Write a program to display many values from XML file using PHP looping statement	
25	04.09.2025	JSON object literal, illustrate the various ways of accessing the object values.	

26	04.09.2025	Program to create a JavaScript array by parsing a JSON string and access the array values	
27	04.09.2025	Create a JSON file using your own data (which include an array element) and access the JSON values by array indexing and looping	
28	05.09.2025	Illustrate the usage of date and function in JSON	
29	05.09.2025	Illustrate the usage of JSON.stringify() on array, date and function	
30	06.09.2025	Program to send HTML form data to JSON file	
31	07.09.2025	Display the data from JSON file to HTML forms	
32	07.09.2025	Program to store HTML form data as a separate JSON file.	
33	07.09.2025	Implement the types of expressions in AngularJS	
34	08.09.2025	Write a code to implement the classes of ng-model directive	
35	08.09.2025	Implement the types of data binding in AngularJS	
36	08.09.2025	Implement AngularJS modules and Controller directives	
37	09.09.2025	Implement any ten AngularJS filters	
38	09.09.2025	Illustrate the services of $http in AngularJS	
39	09.09.2025	Display table using AngularJS ng-repeat directive (without PHP and with PHP). Use necessary CSS.	
40	09.09.2025	Illustrate about Binding application data to the attributes of HTML DOM elements.	
41	24.09.2025	Implement the AngularJS even directives	
42	24.09.2025	Design any user registration form in AngularJS and validate all the fields in the form. Use necessary CSS.	
43	25.09.2025	Implement the usage of core module in Node.JS	
44	08.10.2025	Implement Local module and Third-party Module	
45	08.10.2025	Implementation of HTTP module	
46	08.10.2025	Implementation of methods in file system module	
47	15.10.2025	Illustrate URL module	
48	16.10.2025	Illustrate Event handling in Node.JS	
49	22.10.2025	File uploading in Node.JS	
50	23.10.2025	Sending Email in Node.JS	

EX.NO:01 DATE: 26.7.2025	**Create a HTML page to display your Bio-Data**

AIM:

To write a program Create a HTML page to display your Bio-Data.

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Bio-Data</title>
</head>
<body>
  <h1>John Doe</h1>
  <p><strong>Contact:</strong> john.doe@email.com | (555) 123-4567</p>

  <h2>Summary</h2>
  <p>Highly motivated and results-oriented professional with 5 years of experience in Software Development. Proven ability to deliver high-quality projects on time.</p>

  <h2>Experience</h2>
  <ul>
    <li><strong>Senior Developer</strong> - Tech Solutions Inc. (2020 - Present)
      <ul>
        <li>Led a team of 3 developers on the new e-commerce platform.</li>
        <li>Reduced server response time by 20% through optimization.</li>
      </ul>
    </li>
  </ul>

  <h2>Education</h2>
  <ul>
    <li><strong>M.S. Computer Science</strong> - State University (2018)</li>
    <li><strong>B.E. Information Technology</strong> - City College (2016)</li>
  </ul>

  <h2>Skills</h2>
  <p>Python, JavaScript, HTML, CSS, SQL, AWS, Git</p>
</body>
</html>
```

OUTPUT :

John Doe

Contact: john.doe@email.com | (555) 123-4567

Summary

Highly motivated and results-oriented professional with 5 years of experience in Software Development. Proven ability to deliver high-quality projects on time.

Experience

- **Senior Developer** - Tech Solutions Inc. (2020 - Present)
  - Led a team of 3 developers on the new e-commerce platform.
  - Reduced server response time by 20% through optimization.

Education

- **M.S. Computer Science** - State University (2018)
- **B.E. Information Technology** - City College (2016)

Skills

Python, JavaScript, HTML, CSS, SQL, AWS, Git

RESULT :

Thus the above program was executed successfully.

EX.NO:02 DATE: 26.7.2025	**Create a web page to display the details of any two items**

AIM:

To write a program Create a web page to display the details of any two items.

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
   <title>Item Details</title>
</head>
<body>
   <h1>Featured Products</h1>

   <h2>Product A: Wireless Headphones</h2>
   <p><strong>Price:</strong> $79.99</p>
   <p><strong>Description:</strong> Noise-cancelling over-ear headphones with 20 hours of battery life. Available in Black and White.</p>

   <h2>Product B: Smart Watch</h2>
   <p><strong>Price:</strong> $199.00</p>
   <p><strong>Description:</strong> Tracks fitness, sleep, and heart rate. Features a durable aluminum case and waterproof design.</p>
</body>
</html>
```

OUTPUT :

Featured Products

Product A: Wireless Headphones

Price: $79.99

Description: Noise-cancelling over-ear headphones with 20 hours of battery life. Available in Black and White.

Product B: Smart Watch

Price: $199.00

Description: Tracks fitness, sleep, and heart rate. Features a durable aluminum case and waterproof design.

RESULT :

Thus the above program was executed successfully.

EX.NO:03 DATE: 28.7.2025	**Implement all text and image formatting tags**

AIM:

To write a program Implement all text and image formatting tags.

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formatting Tags Demo</title>
</head>
<body>
  <h1>Text and Image Formatting Tags</h1>

  <p>This is a paragraph of **normal text**.</p>

  <p>Here we show **bold** (using &lt;b&gt; and &lt;strong&gt;), <i>italic</i> (using &lt;i&gt; and &lt;em&gt;), <mark>marked/highlighted</mark>, <small>small print</small>, and <ins>inserted</ins> text.</p>

  <p>Math Example: $x^{2}$ is $\text{superscript}$ ($\text{&lt;sup&gt;}$), and $\text{H}_{2}\text{O}$ is $\text{subscript}$ ($\text{&lt;sub&gt;}$).</p>

  <p>The product is <del>discontinued</del> now. This is a <code>short code snippet</code>.</p>

  <hr>

  <h2>Image Tag Example</h2>
  <img src="https://via.placeholder.com/300x150" alt="Placeholder Image" style="border: 2px solid blue; padding: 5px; width: 50%;">
  <p>The image above is set to have a **blue border**, some internal **padding**, and a **width of 50%** of the page using the `style` attribute (inline CSS).</p>
</body>
</html>
```

OUTPUT :

Text and Image Formatting Tags

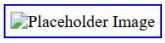
This is a paragraph of **normal text**.

Here we show **bold** (using <b> and), *italic* (using <i> and), marked/highlighted, small print, and inserted text.

Math Example: x^{2} is superscript (<sup>), and H_{2}O is subscript (<sub>).

The product is ~~discontinued~~ now. This is a `short code snippet`.

Image Tag Example



The image above is set to have a **blue border**, some internal **padding**, and a **width of 50%** of the page using the `style` attribute (inline CSS).

RESULT :

Thus the above program was executed successfully.

EX.NO:04 DATE: 28.7.2025	Design any web page using Frames

AIM:

To write a program Design any web page using Frames.

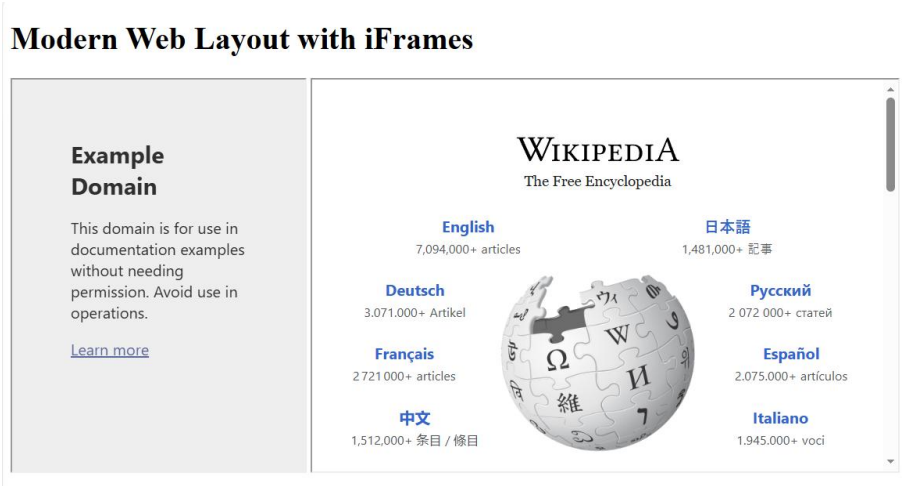
PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
  <title>iFrame Example</title>
</head>
<body>
  <h1>Modern Web Layout with iFrames</h1>

  <iframe src="https://www.example.com" width="300" height="400" title="Example Website 1"></iframe>

  <iframe src="https://www.wikipedia.org" width="600" height="400" title="Example Website 2"></iframe>
</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:05 DATE: 29.7.2025	Implement HTML tables and Lists

AIM:

To write a program Implement HTML tables and Lists.

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
  <title>Tables and Lists Demo</title>
</head>
<body>
  <h1>Table and List Examples</h1>

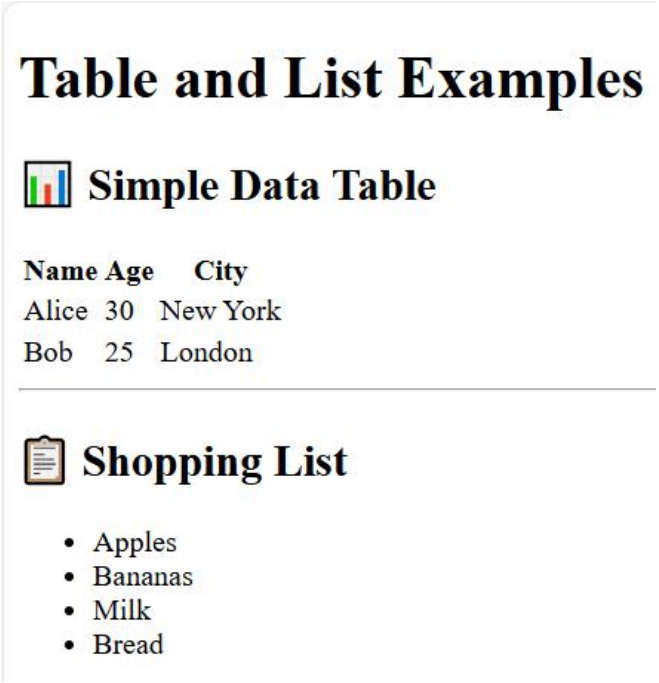
  <h2>  Simple Data Table</h2>
  <table>
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th>City</th>
    </tr>
    <tr>
      <td>Alice</td>
      <td>30</td>
      <td>New York</td>
    </tr>
    <tr>
      <td>Bob</td>
      <td>25</td>
      <td>London</td>
    </tr>
  </table>

  <hr>

  <h2>  Shopping List</h2>
  <ul>
    <li>Apples</li>
    <li>Bananas</li>
    <li>Milk</li>
    <li>Bread</li>
  </ul>

</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:06 DATE: 29.7.2025	**Illustrate Cascading Style Sheet (inline, internal, external and multiple CSS)**

AIM:

To write a program Illustrate Cascading Style Sheet (inline, internal, external and multiple CSS)

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
   <title>CSS Implementation Demo</title>
   <link rel="stylesheet" href="external-styles.css">
   <link rel="stylesheet" href="multiple-styles.css">

   <style>
      h1 {
         color: purple; /* Internal style for H1 */
         border-bottom: 2px solid purple;
      }
      .internal-style {
         background-color: lightyellow;
         padding: 10px;
      }
   </style>
</head>
<body>
   <h1>CSS Implementation Methods</h1>

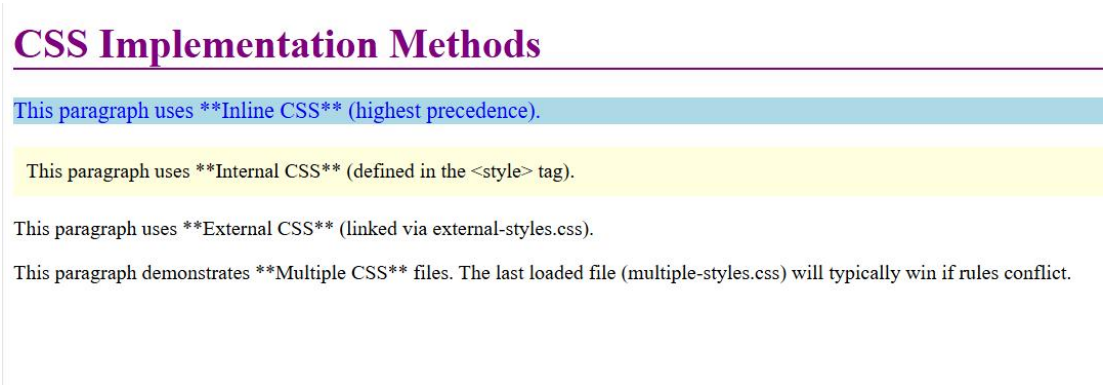
   <p style="color: blue; font-size: 1.1em; background-color: lightblue;">
      This paragraph uses **Inline CSS** (highest precedence).
   </p>

   <p class="internal-style">
      This paragraph uses **Internal CSS** (defined in the &lt;style&gt; tag).
   </p>

   <p id="external-style">
      This paragraph uses **External CSS** (linked via external-styles.css).
   </p>

   <p class="multiple-style">
      This paragraph demonstrates **Multiple CSS** files. The last loaded file (multiple-styles.css) will typically win if rules conflict.
   </p>
</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:07 DATE: 30.7.2025	**Design any user registration form using all HTML input elements along with CSS**

AIM:

To write a program Design any user registration form using all HTML input elements along with CSS

PROGRAM:

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width,initial-scale=1" />
<title>Simple Registration</title>
<style>
 /* Container */
 body {
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, Arial;
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: 100vh;
  margin: 0;
  background: linear-gradient(135deg,#f7f9fc,#e9f0ff);
 }

 /* Card */
 .card {
  width: 320px;
  padding: 22px;
  border-radius: 12px;
  box-shadow: 0 6px 22px rgba(20,40,80,0.08);
  background: #fff;
 }

 h2 { margin: 0 0 12px; font-size: 20px; color:#183056; text-align:center; }

 /* Inputs */
 .field { display:flex; flex-direction:column; gap:6px; margin-bottom:12px; }
 label { font-size:13px; color:#415a72; }
 input {
  padding:10px 12px;
  border:1px solid #d6e0ef;
  border-radius:8px;
  font-size:14px;
  outline:none;
  transition: box-shadow .12s, border-color .12s;
```

```
  }
  input:focus { border-color:#6b8cff; box-shadow:0 4px 14px rgba(107,140,255,0.12); }

  /* Button */
  button {
    width:100%;
    padding:10px;
    border: none;
    border-radius:10px;
    background: linear-gradient(90deg,#556bff,#6b8cff);
    color:#fff;
    font-weight:600;
    cursor:pointer;
    font-size:15px;
  }
  button:active { transform: translateY(1px); }

  /* Small helper text */
  .muted { font-size:12px; color:#7b8aa0; text-align:center; margin-top:10px; }
</style>
</head>
<body>

  <form class="card" onsubmit="event.preventDefault(); alert('Registered:\\n' +
    'Name: ' + name.value + '\\nEmail: ' + email.value);">
    <h2>Create an account</h2>

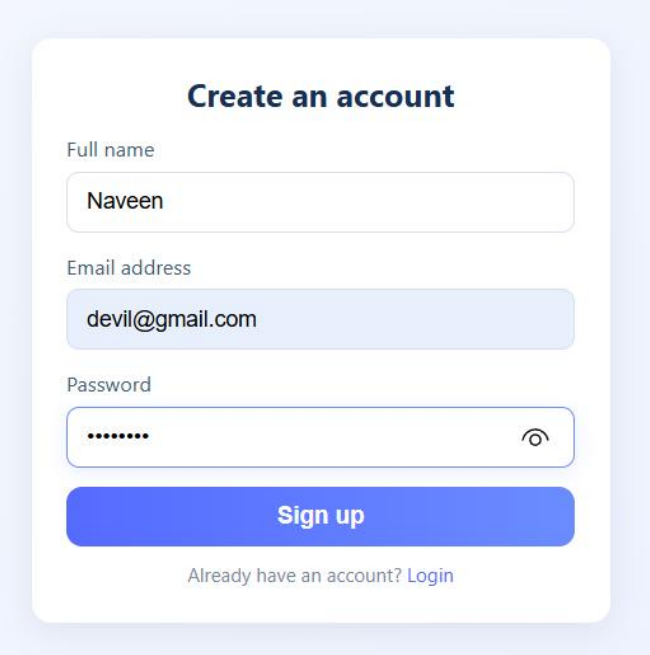
    <div class="field">
      <label for="name">Full name</label>
      <input id="name" name="name" type="text" placeholder="Your name" required />
    </div>

    <div class="field">
      <label for="email">Email address</label>
      <input id="email" name="email" type="email" placeholder="you@example.com"
required />
    </div>

    <div class="field">
      <label for="password">Password</label>
      <input id="password" name="password" type="password" placeholder="Create a
password" minlength="6" required />
    </div>

    <button type="submit">Sign up</button>
    <div class="muted">Already have an account? <a href="#" style="color:#556bff;text-
decoration:none;">Login</a></div>
  </form>
</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:08 DATE: 30.7.2025	**Display the Emoji symbols using HTML**

AIM:

To write a program Display the Emoji symbols using HTML.

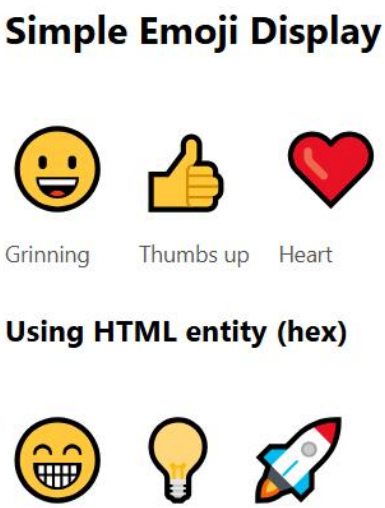
PROGRAM:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Emoji Demo</title>
  <style>
    body{font-family:system-ui,Segoe UI,Roboto,Arial;margin:32px;line-height:1.8}
    .row{display:flex;gap:18px;align-items:center;flex-wrap:wrap}
    .emoji{font-size:48px}
    .label{font-size:14px;color:#555}
  </style>
</head>
<body>
  <h2>Simple Emoji Display</h2>

  <!-- Direct emoji characters -->
  <div class="row">
    <div><div class="emoji" aria-label="grinning face">  </div><div class="label">Grinning</div></div>
    <div><div class="emoji" aria-label="thumbs up">  </div><div class="label">Thumbs up</div></div>
    <div><div class="emoji" aria-label="red heart">  </div><div class="label">Heart</div></div>
  </div>

  <!-- Unicode code point (hex) -->
  <h3>Using HTML entity (hex)</h3>
  <div class="row">
    <div class="emoji" aria-hidden="true">&#x1F601;</div> <!--  -->
    <div class="emoji" aria-hidden="true">&#x1F4A1;</div> <!--  -->
    <div class="emoji" aria-hidden="true">&#x1F680;</div> <!--  -->
  </div>
</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:09 DATE: 30.7.2025	Create menus and illustrate drawing borders, shading and shapes in HTML

AIM:

To write a program Create menus and illustrate drawing borders, shading and shapes in HTML.

PROGRAM:

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Simple Demo</title>
<style>
  body{font-family:Arial;margin:24px;background:#f7f9fc;color:#122;}
  nav{display:flex;gap:10px;align-items:center;margin-bottom:16px}
  a{padding:6px 10px;text-decoration:none;color:#345;border-radius:6px}
  a:hover{background:#e8f3ff}
  /* accessible dropdown using details */
  details{position:relative}
  summary{list-style:none;cursor:pointer;padding:6px 10px;border-radius:6px}
  /* card with border + shadow */
  .card{display:inline-block;padding:12px;border-radius:8px;border:1px solid #d7e3fb;
       background:linear-gradient(180deg,#fff,#f8fbff);box-shadow:0 8px 18px
rgba(16,40,80,.06);margin-right:12px}
  .row{display:flex;gap:10px;align-items:center;margin-top:10px}
  .box{width:56px;height:56px;display:grid;place-items:center;border-
radius:8px;background:#fff;border:3px solid}
  .solid{border-color:#d9534f}
  .dashed{border-style:dashed;border-color:#f0ad4e}
  .circle{width:70px;height:70px;border-radius:50%;background:linear-
gradient(135deg,#ff9,#f90)}
  /* triangle */
  .tri{width:0;height:0;border-left:30px solid transparent;border-right:30px solid transparent;
      border-bottom:56px solid #6c9}
  small{display:block;color:#667;margin-top:8px;font-size:13px}
</style>
</head>
<body>

<nav>
  <strong style="color:#246;padding-right:6px">Logo</strong>
  <a href="#">Home</a>
  <a href="#">About</a>
  <details>
    <summary>Menu ▾ </summary>
```

```
    <div    style="padding:8px;background:#fff;border-radius:8px;box-shadow:0    8px    18px
rgba(0,0,0,.06);margin-top:6px">
      <a href="#" style="display:block;padding:6px 8px;color:#234">Item 1</a>
      <a href="#" style="display:block;padding:6px 8px;color:#234">Item 2</a>
    </div>
  </details>
  <a href="#">Contact</a>
</nav>

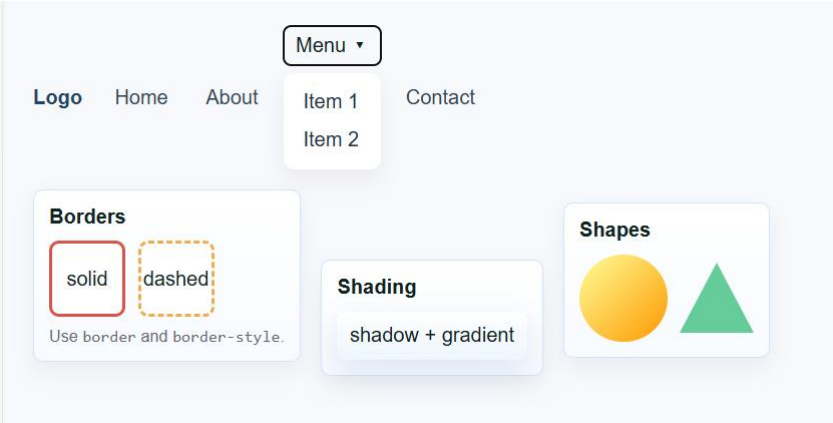
<div class="card">
  <strong>Borders</strong>
  <div class="row">
    <div class="box solid">solid</div>
    <div class="box dashed">dashed</div>
  </div>
  <small>Use <code>border</code> and <code>border-style</code>.</small>
</div>

<div class="card">
  <strong>Shading</strong>
  <div class="row" style="margin-top:10px">
    <div          style="padding:10px;border-radius:8px;box-shadow:0          10px          24px
rgba(36,70,140,.12);background:linear-gradient(180deg,#fff,#eef6ff)">
      shadow + gradient
    </div>
  </div>
</div>

<div class="card">
  <strong>Shapes</strong>
  <div class="row" style="margin-top:10px">
    <div class="circle" title="circle"></div>
    <div class="tri" title="triangle"></div>
  </div>
</div>

</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:10 DATE: 02.08.2025	**Implement all web APIs (drag and drop, Geolocation, Third Party, JavaScript Validation, Web History, Storage, Web Worker)**

AIM:

To write a program Implement all web APIs (drag and drop, Geolocation, Third Party, JavaScript Validation, Web History, Storage, Web Worker) .

PROGRAM:

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Mini Web APIs Demo</title>
<style>
body{font-family:Arial;margin:18px;background:#f7fbff;color:#123}
.box{width:80px;height:80px;background:#6bbcff;border-radius:8px;display:grid;place-items:center;color:#fff;cursor:grab}
.drop{width:200px;height:100px;border:2px dashed #9ab;border-radius:8px;display:grid;place-items:center;margin-top:8px}
.row{display:flex;gap:12px;flex-wrap:wrap}
.card{background:#fff;padding:10px;border-radius:8px;box-shadow:0 6px 18px rgba(0,30,80,.06);min-width:240px}
small{color:#556;font-size:12px}
</style>
</head>
<body>

<h3>Compact Web APIs demo</h3>

<div class="row">
  <!-- Drag & Drop -->
  <div class="card">
    <strong>Drag & Drop</strong>
    <div id="drag" class="box" draggable="true">DRAG</div>
    <div id="drop" class="drop">Drop here</div>
    <small id="dmsg"></small>
  </div>

  <!-- Geolocation -->
  <div class="card">
    <strong>Geolocation</strong>
    <button id="geoBtn">Get Location</button>
    <div id="loc">—</div>
    <small>Uses navigator.geolocation</small>
  </div>
```

```
  <!-- Third-party (Lodash) -->
  <div class="card">
    <strong>Third-party (Lodash)</strong>
    <input id="txt" placeholder="type name"><button id="cap">Cap</button>
    <div id="capout">—</div>
    <small>CDN script + _.capitalize</small>
  </div>

  <!-- Validation + Storage -->
  <div class="card">
    <strong>Validation & Storage</strong>
    <form id="f">
      <input id="name" placeholder="name (letters only)" required pattern="^[A-Za-
z ]+$"><br><br>
      <input id="age" placeholder="age (18-99)" required pattern="^(1[89]|[2-9][0-
9])$"><br><br>
      <button>Submit</button>
    </form>
    <div><button id="save">Save name to localStorage</button> <button
id="load">Load</button></div>
    <div id="stored">—</div>
  </div>

  <!-- History API -->
  <div class="card">
    <strong>History API</strong>
    <div id="page">Page A</div>
    <button id="goB">Go to B</button>
    <small>Use browser Back to return</small>
  </div>

  <!-- Web Worker -->
  <div class="card">
    <strong>Web Worker</strong>
    <button id="startw">Start Worker (calc)</button>
    <div id="wout">—</div>
    <small>Worker sums 1..N in background</small>
  </div>
</div>

<!-- Lodash CDN (third-party) -->
<script src="https://cdn.jsdelivr.net/npm/lodash@4.17.21/lodash.min.js"></script>

<script>
// --- Drag & Drop ---
const drag = document.getElementById('drag'), drop = document.getElementById('drop'),
dmsg = document.getElementById('dmsg');
drag.addEventListener('dragstart', e => e.dataTransfer.setData('text/plain','dragged'));
drop.addEventListener('dragover', e => e.preventDefault());
drop.addEventListener('drop', e => { e.preventDefault(); dmsg.textContent = 'Dropped!'; });
```

```
// --- Geolocation ---
document.getElementById('geoBtn').onclick = () => {
  const out = document.getElementById('loc');
  if (!navigator.geolocation) return out.textContent = 'Not supported';
  navigator.geolocation.getCurrentPosition(p => out.textContent = `Lat:
${p.coords.latitude.toFixed(4)}, Lon: ${p.coords.longitude.toFixed(4)}` ,
    err => out.textContent = 'Denied / error');
};

// --- Lodash usage (third-party) ---
document.getElementById('cap').onclick = () => {
  const v = document.getElementById('txt').value || '';
  document.getElementById('capout').textContent = _.capitalize(v);
};

// --- Validation + Storage ---
const form = document.getElementById('f');
form.addEventListener('submit', e => {
  e.preventDefault();
  if (!form.checkValidity()) { alert('Fix fields'); return; }
  alert('Form OK — name: ' + document.getElementById('name').value);
});
document.getElementById('save').onclick = () => {
  const n = document.getElementById('name').value;
  if(n) localStorage.setItem('demo_name', n);
  document.getElementById('stored').textContent = 'Saved';
};
document.getElementById('load').onclick = () => {
  document.getElementById('stored').textContent = localStorage.getItem('demo_name') || 'No
name';
};

// --- History API ---
const page = document.getElementById('page');
document.getElementById('goB').onclick = () => {
  history.pushState({page:'B'}, 'B', '?page=B');
  page.textContent = 'Page B';
};
window.addEventListener('popstate', e => {
  page.textContent = (e.state && e.state.page==='B') ? 'Page B' : 'Page A';
});

// --- Web Worker via Blob ---
const wout = document.getElementById('wout');
document.getElementById('startw').onclick = () => {
  // small worker that sums numbers up to N (N=1000000)
  const code = `self.onmessage=e=>{const n=e.data;let s=0;for(let
i=1;i<=n;i++)s+=i;self.postMessage(s)}`;
  const worker = new Worker(URL.createObjectURL(new
Blob([code],{type:'text/javascript'})));
```

```
 worker.onmessage = m => { wout.textContent = 'Sum=' + m.data; worker.terminate(); };
  wout.textContent = 'Working...'; worker.postMessage(1000000);
};

</script>
</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:11 DATE:3.8.25	User Registration Form with JavaScript Validation

AIM:

To write a program for User Registration Form with JavaScript Validation.

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
<title>Registration Form</title>
</head>
<body>

<h2>User Registration</h2>

<form onsubmit="return checkForm()">
   Name: <input type="text" id="name"><br><br>
   Email: <input type="text" id="email"><br><br>
   Password: <input type="password" id="pass"><br><br>
   <input type="submit" value="Register">
</form>

<script>
function checkForm() {
   let name = document.getElementById("name").value;
   let email = document.getElementById("email").value;
   let pass = document.getElementById("pass").value;

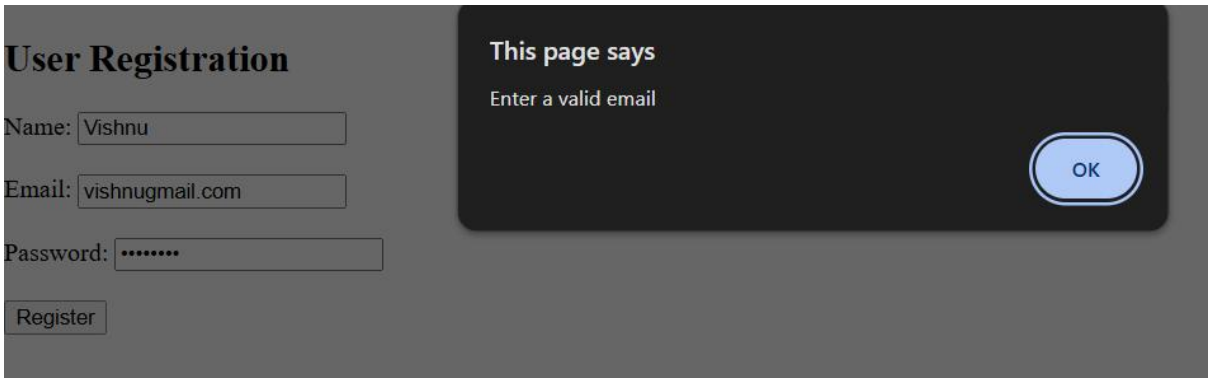
   if (name == "" || email == "" || pass == "") {
      alert("All fields are required");
      return false;
   }

   if (!email.includes("@")) {
      alert("Enter a valid email");
      return false;
   }

   return true;
}
</script>

</body>
</html>
```

OUTPUT:



RESULT:

The above program was executed successfully.

EX.NO:12 DATE:3.8.25	Simple Webpage using HTML + CSS + JS + PHP + MySQL

AIM:

To write a program to create a simple webpage using HTML, CSS, PHP and MySQL.

PROGRAM:

Create MySQL Database & Table:

```
CREATE DATABASE awpdb;

USE awpdb;

CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50),
   email VARCHAR(50)
);
```

index.html:

```
<!DOCTYPE html>
<html>
<head>
<title>User Form</title>

<style>
body {
   background: #f3f3f3;
   font-family: Arial;
}

.container {
   width: 350px;
   background: white;
   padding: 20px;
   margin: 50px auto;
   border-radius: 10px;
   box-shadow: 0 0 10px gray;
}

input {
   width: 95%;
```

```
    padding: 10px;
    margin-top: 10px;
    border-radius: 5px;
    border: 1px solid gray;
}

button {
    padding: 10px;
    width: 100%;
    background: blue;
    color: white;
    border: none;
    margin-top: 15px;
    border-radius: 5px;
    cursor: pointer;
}

button:hover {
    background: darkblue;
}
</style>

<script>
function validateForm() {
    let name = document.getElementById("name").value;
    let email = document.getElementById("email").value;

    if (name == "" || email == "") {
        alert("Both fields are required!");
        return false;
    }

    if (!email.includes("@")) {
        alert("Enter a valid email ID!");
        return false;
    }

    return true;
}
</script>

</head>
<body>
```

```
<div class="container">
<h2>Save User</h2>

<form action="save.php" method="post" onsubmit="return validateForm()">
   <input type="text" id="name" name="name" placeholder="Enter Name">
   <input type="text" id="email" name="email" placeholder="Enter Email">
   <button type="submit">Save</button>
</form>
</div>

</body>
</html>
```

save.php:

```
<?php

$con = mysqli_connect("localhost", "root", "", "awpdb");

$name = $_POST["name"];
$email = $_POST["email"];

$sql = "INSERT INTO users(name, email) VALUES('$name', '$email')";
mysqli_query($con, $sql);

echo "<h2>User Saved Successfully!</h2>";
echo "<a href='index.html'>Go Back</a>";

?>
```

OUTPUT:



RESULT:

The above program was executed successfully.

EX.NO:13 DATE:3.8.25	Simple Webpage using HTML + CSS + JS + PHP + MySQL

AIM:

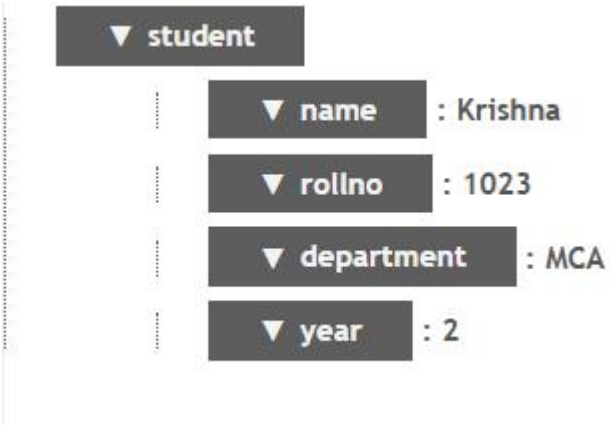
To write a XML program to create a XML file.

PROGRAM:

Students.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<student>
   <name>Krishna</name>
   <rollno>1023</rollno>
   <department>MCA</department>
   <year>2</year>
</student>
```

OUTPUT:



RESULT:

The above program was executed successfully.

EX.NO:14 DATE:3.8.25	XML with Internal DTD

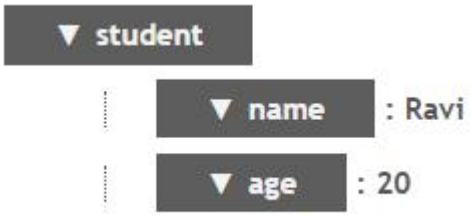
AIM:

To write a program to illustrate XML with Internal DTD.

PROGRAM:

```
<!DOCTYPE student [
   <!ELEMENT student (name, age)>
   <!ELEMENT name (#PCDATA)>
   <!ELEMENT age (#PCDATA)>
]>
<student>
   <name>Ravi</name>
   <age>20</age>
</student>
```

OUTPUT:



RESULT:

The above program was executed successfully.

EX.NO:15 DATE:4.8.25	XML with External DTD

AIM:

To write a program to illustrate XML with External DTD.

PROGRAM:

student.dtd:

```
<!ELEMENT student (name, age)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT age (#PCDATA)>
```

student.xml:

```
<!DOCTYPE student SYSTEM "student.dtd">
<student>
   <name>Ravi</name>
   <age>20</age>
</student>
```

OUTPUT:



RESULT:

The above program was executed successfully.

EX.NO:16 DATE:4.8.25	Display XML Table in HTML Document

AIM:

To write a program to display XML Table in HTML Document.

PROGRAM:

data.xml:

```
<items>
   <item>
      <name>Pen</name>
      <price>10</price>
   </item>
   <item>
      <name>Book</name>
      <price>40</price>
   </item>
</items>
```

display.html:

```
<!DOCTYPE html>
<html>
<body>
<h2>Items List</h2>
<table border="1" id="tbl"></table>
<script>
var x = new XMLHttpRequest();
x.open("GET", "data.xml", false);
x.send();
var xml = x.responseXML;
var items = xml.getElementsByTagName("item");
let out = "<tr><th>Name</th><th>Price</th></tr>";
for (let i = 0; i < items.length; i++) {
   let name = items[i].getElementsByTagName("name")[0].textContent;
   let price = items[i].getElementsByTagName("price")[0].textContent;
   out += "<tr><td>" + name + "</td><td>" + price + "</td></tr>";
}
document.getElementById("tbl").innerHTML = out;
</script>
</body></html>
```

OUTPUT:

Items List

Name	Price
Pen	10
Book	40

RESULT:

The above program was executed successfully.

EX.NO:17 DATE:4.8.25	Send HTML Form Data to XML (Using JavaScript)

AIM:

To write a program to send HTML form data to XML using JavaScript.

PROGRAM:

index.html:

```
<!DOCTYPE html>
<html>
<head>
<title>Send Form Data to XML</title>
</head>
<body>

<h2>Send HTML Form Data to XML File</h2>

<form onsubmit="sendData(); return false;">
   Name: <input type="text" id="name"><br><br>
   Email: <input type="text" id="email"><br><br>
   <button type="submit">Save to XML</button>
</form>

<h3>XML Created:</h3>
<pre id="output"></pre>

<script>
function sendData() {
   let name = document.getElementById("name").value;
   let email = document.getElementById("email").value;

   // Create XML string
   let xml =
      "<user>\n" +
      "  <name>" + name + "</name>\n" +
      "  <email>" + email + "</email>\n" +
      "</user>";

   document.getElementById("output").textContent = xml;

   // Send XML to PHP for saving
   let x = new XMLHttpRequest();
```

```
    x.open("POST", "savexml.php", true);
    x.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
    x.send("xmlData=" + encodeURIComponent(xml));
}
</script>
</body>
</html>
```

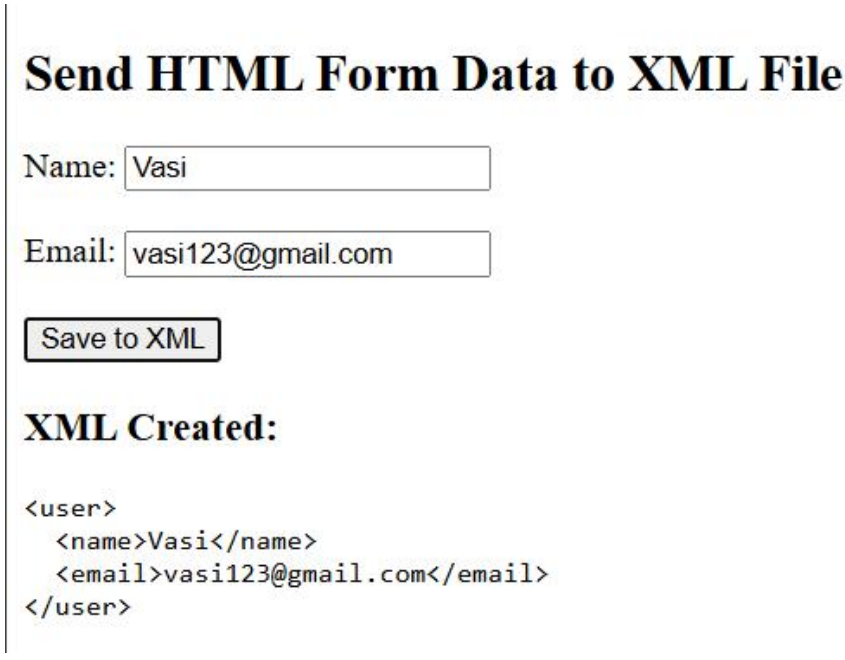
savexml.php:

```
<?php
$xml = $_POST["xmlData"];

// Save the XML string into a file
file_put_contents("user.xml", $xml);

echo "XML File Saved Successfully!";
?>
```

OUTPUT:



RESULT:

The above program was executed successfully.

EX.NO:18 DATE:4.8.25	Display XML Data into HTML Form

AIM:

To write a program to display XML data into HTML form.

PROGRAM:

user.xml:

```
<user>
   <name>Raja</name>
</user>
```

form.html:

```
<!DOCTYPE html>
<html>
<body>
Name: <input type="text" id="name">
<script>
var x = new XMLHttpRequest();
x.open("GET", "user.xml", false);
x.send();
var xml = x.responseXML;
document.getElementById("name").value =
   xml.getElementsByTagName("name")[0].textContent;
</script>
</body>
</html>
```

OUTPUT:



RESULT:

The above program was executed successfully.

EX.NO:19 DATE:4.8.25	XML Schema (XSD) for Simple Data Type

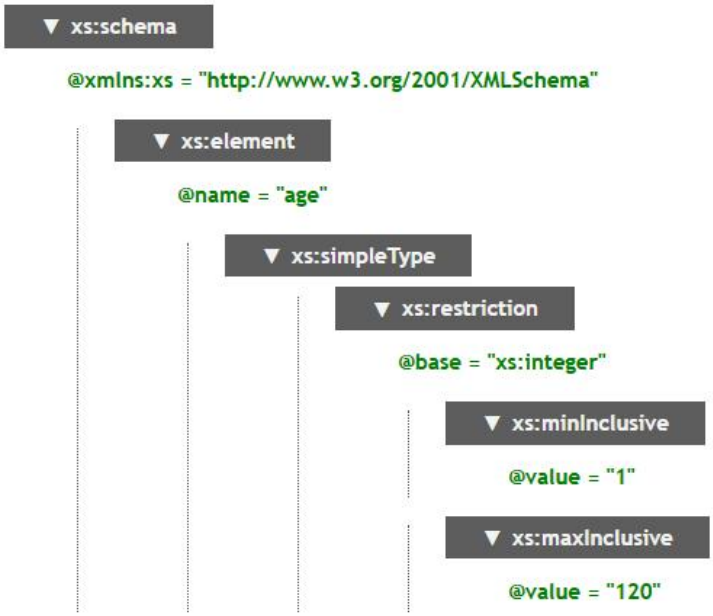
AIM:

To write a program to create an XML schema for simple data type.

PROGRAM:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="age">
    <xs:simpleType>
      <xs:restriction base="xs:integer">
        <xs:minInclusive value="1"/>
        <xs:maxInclusive value="120"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```

OUTPUT:



RESULT:

The above program was executed successfully.

EX.NO:20 DATE:4.8.25	XML Schema (XSD) for Complex Data Type

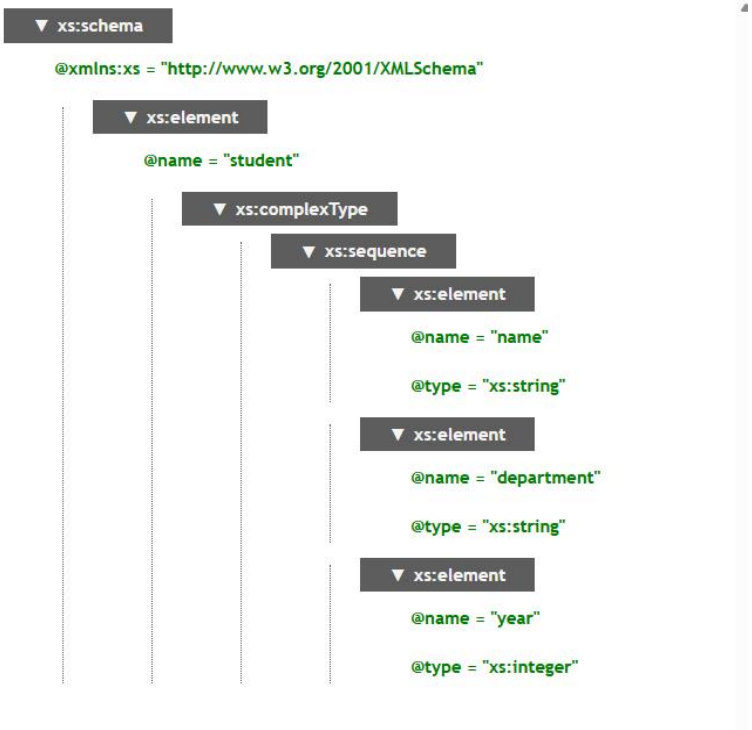
AIM:

To write a program to create an XML schema for Complex Data Type.

PROGRAM:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="student">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="department" type="xs:string"/>
        <xs:element name="year" type="xs:integer"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

OUTPUT:



RESULT:

The above program was executed successfully.

EX.NO:21 DATE: 06.8.2025	**Write a program to read XML data using PHP simplexml_load_string() function**

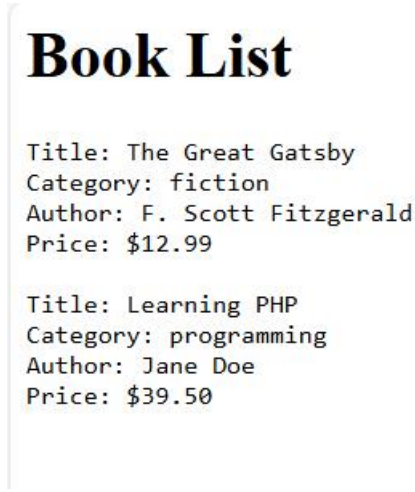
AIM:

To write a program to read XML data using PHP simpleXML_load_string() function

PROGRAM:

```
<?php
 $xmlString = <<<XML
 <bookstore>
 <book category="fiction">
 <title lang="en">The Great Gatsby</title>
 <author>F. Scott Fitzgerald</author>
 <year>1925</year>
 <price>12.99</price>
 </book>
 <book category="programming">
 <title lang="en">Learning PHP</title>
 <author>Jane Doe</author>
 <year>2023</year>
 <price>39.50</price>
 </book>
 </bookstore>
 XML;
 $xml = simplexml_load_string($xmlString);
 if ($xml !== false) {
 echo "<h1>Book List</h1><pre>";
 foreach ($xml->book as $book) {
 echo "Title: " . (string)$book->title . "\n";
 echo "Category: " . (string)$book['category'] . "\n";
 echo "Author: " . (string)$book->author . "\n";
 echo "Price: $" . (string)$book->price . "\n\n";
 }
 echo "</pre>";
 } else {
 echo "Failed to load XML.";
 }
 ?>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:22 DATE: 06.8.2025	**Write a program to read data stored in XML file using PHP**

AIM:

To write a program to read data stored in XML file using PHP

PROGRAM:

```
<?php
 $xmlData = <<<XML
 <?xml version="1.0" encoding="UTF-8"?>
 <user>
 <name>Devil Kumar</name>
 <email>devil.kumar@example.com</email>
 <city>Madurai</city>
 </user>
 XML;
 $xml = simplexml_load_string($xmlData);
 echo "Name: " . $xml->name . "<br>";
 echo "Email: " . $xml->email . "<br>";
 echo "City: " . $xml->city . "<br>";
 ?>
```

OUTPUT :

Name: Devil Kumar
Email: devil.kumar@example.com
City: Madurai

RESULT :

Thus the above program was executed successfully.

EX.NO:23 DATE: 24.8.2025	**Write a program to display the XML data using PHP**

AIM:

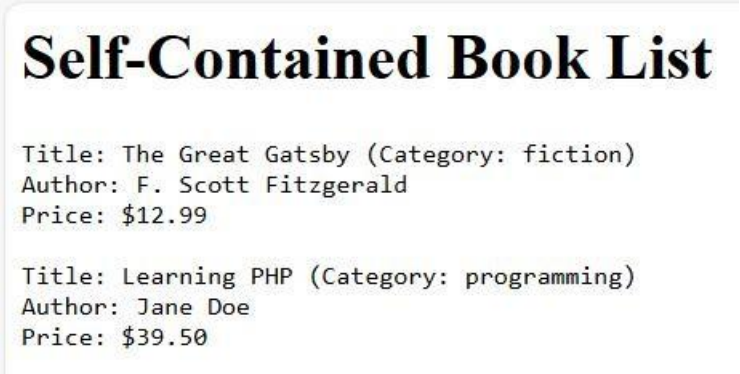
To write a program to display the XML data using PHP.

PROGRAM:

```
<?php
$xmlString = <<<XML
<bookstore>
    <book category="fiction">
        <title lang="en">The Great Gatsby</title>
        <author>F. Scott Fitzgerald</author>
        <price>12.99</price>
    </book>
    <book category="programming">
        <title lang="en">Learning PHP</title>
        <author>Jane Doe</author>
        <price>39.50</price>
    </book>
</bookstore>
XML;
$xml = simplexml_load_string($xmlString);
if ($xml !== false) {
    echo "<h1>Self-Contained Book List</h1><pre>";
    foreach ($xml->book as $book) {
        echo "Title: " . (string)$book->title . " (Category:
 " . (string)$book['category'] . ")\n";
        echo "Author: " . (string)$book->author . "\n";
        echo "Price: $" . (string)$book->price . "\n\n";
```

```
    }
    echo "</pre>";
} else {
    echo "Failed to load XML from string.";
}
?>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:24 DATE: 29.8.2025	**Write a program to display many values from XML file using PHP looping statement**

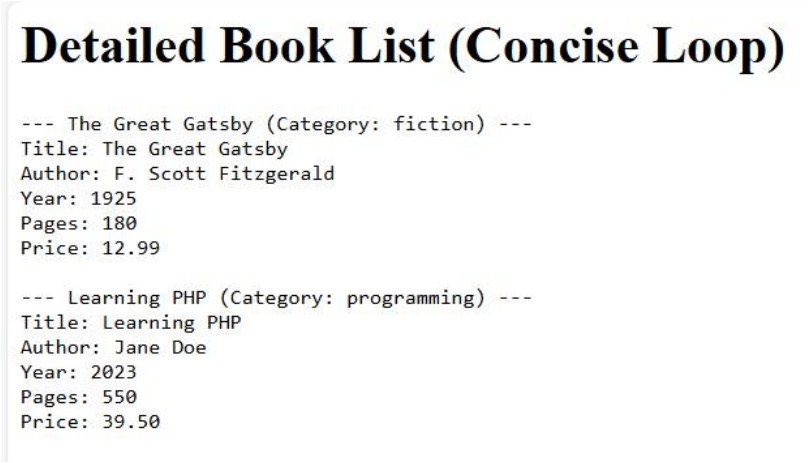
AIM:

To write a program to display many values from the XML file using PHP looping statement.

PROGRAM:

```
<?php
$xmlString = <<<XML
<bookstore>
   <book category="fiction">
      <title lang="en">The Great Gatsby</title>
      <author>F. Scott Fitzgerald</author>
      <year>1925</year>
      <pages>180</pages>
      <price>12.99</price>
   </book>
   <book category="programming">
      <title lang="en">Learning PHP</title>
      <author>Jane Doe</author>
      <year>2023</year>
      <pages>550</pages>
      <price>39.50</price>
   </book>
</bookstore>
XML;
$xml = simplexml_load_string($xmlString);
if ($xml !== false) {
   echo "<h1>Detailed Book List (Concise Loop)</h1><pre>";
   foreach ($xml->book as $book) {
      echo "--- " . (string)$book->title . " (Category: " . (string)$book['category'] . ") ---\n";
      foreach ($book as $key => $value) {
         echo ucfirst($key) . ": " . (string)$value . "\n";
      }
      echo "\n";
   }
   echo "</pre>";
} else {
   echo "Failed to load XML from string.";
}
?>
```

OUTPUT :



Detailed Book List (Concise Loop)

```
--- The Great Gatsby (Category: fiction) ---
Title: The Great Gatsby
Author: F. Scott Fitzgerald
Year: 1925
Pages: 180
Price: 12.99

--- Learning PHP (Category: programming) ---
Title: Learning PHP
Author: Jane Doe
Year: 2023
Pages: 550
Price: 39.50
```

RESULT :

Thus the above program was executed successfully.

EX.NO:25 DATE: 04.9.2025	**Create a JavaScript object using JSON object literal, also illustrate the various ways of accessing the object values.**

AIM:

To write a program to create a JavaScript object using json object literal,also Illustrate the various ways of accessing the object values.

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Object Literal Demo</title>
  <style>
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      margin: 20px;
      background-color: #f4f4f9;
    }
    .container {
      max-width: 600px;
      margin: 0 auto;
      background-color: #fff;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
    }
    h2 {
      color: #333;
      border-bottom: 2px solid #007bff;
      padding-bottom: 10px;
    }
    .code-box {
      background-color: #e9ecef;
      padding: 15px;
      border-radius: 4px;
      margin-bottom: 20px;
      overflow-x: auto;
    }
    .output-section p {
      font-size: 1.1em;
      margin: 10px 0;
    }
    .highlight {
      font-weight: bold;
      color: #28a745;
    }
  </style>
</head>
```

```
<body>

   <div class="container">
      <h2>JavaScript Object Access Demonstration</h2>

      <p>The JavaScript Object (stored in the variable <code>book</code>):</p>
      <div class="code-box">
         <pre><code>
const book = {
   title: "The Great Gatsby",
   author: "F. Scott Fitzgerald",
   year: 1925,
   "publisher-city": "New York", // Property with a hyphen
   keywords: ["jazz", "wealth", "tragedy"]
};
         </code></pre>
      </div>

      <hr>

      <div class="output-section">
         <h2>Accessing Object Values</h2>

         <p>Title (Dot Notation): <span id="output-title" class="highlight"></span></p>
         <p>Author (Dot Notation): <span id="output-author" class="highlight"></span></p>

         <p>Publisher City (Bracket Notation - for hyphens): <span id="output-publisher"
         class="highlight"></span></p>
         <p>First Keyword (Bracket Notation - accessing array): <span id="output-keyword"
           class="highlight"></span></p>
         <p>Year (Dot Notation for operation): <span id="output-operation"
           class="highlight"></span></p>
      </div>
     </div>

     <script>

      const book = {
         title: "The Great Gatsby",
         author: "F. Scott Fitzgerald",
         year: 1925,
         "publisher-city": "New York",
         keywords: ["jazz", "wealth", "tragedy"]
      };
      const bookTitle = book.title;

      const bookAuthor = book.author

      const publisherCity = book["publisher-city"];

      const firstKeyword = book.keywords[0];
```

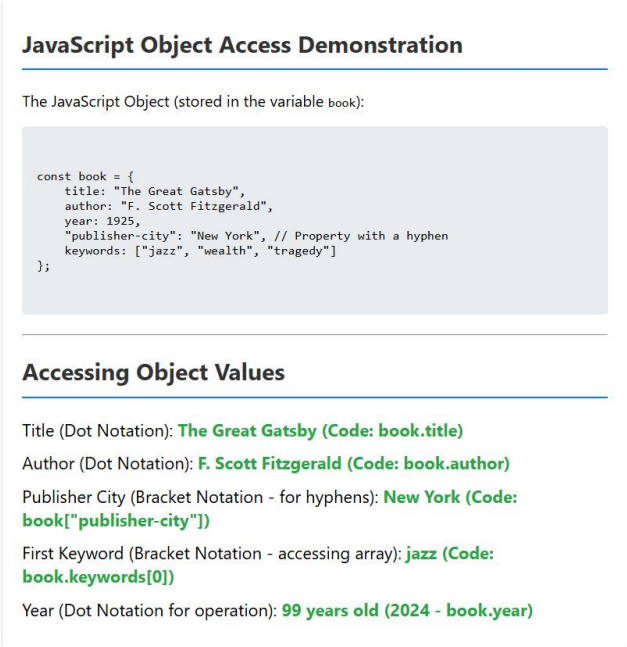
```
const age = 2024 - book.year;

document.getElementById('output-title').textContent = bookTitle + " (Code: book.title)";
document.getElementById('output-author').textContent = bookAuthor + " (Code: book.author)";
document.getElementById('output-publisher').textContent = publisherCity + " (Code:
    book[\"publisher-city\"])";
document.getElementById('output-keyword').textContent = firstKeyword + " (Code:
    book.keywords[0])";
document.getElementById('output-operation').textContent = age + " years old (2024 -
    book.year)";

</script>

</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:26 DATE: 04.9.2025	**Program to create a JavaScript array by parsing a JSON string and access the array values**

AIM:

To write a program to create a JavaScript array by parsing a JSON string and Access the array values.

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
<title>JSON to Array Demo</title>
</head>
  <body>

 <h1>Parsing JSON String to JavaScript Array</h1>

   <p>JSON String (input): <code id="json-string"></code></p>
   <p>Parsed Array Length: <span id="array-length"></span></p>
   <p>First Element (Index 0): <span id="first-element"></span></p>
   <p>Last Element (Index 2): <span id="last-element"></span></p>

   <script>

      const jsonString = '[ "Apple", "Banana", "Cherry", "Date" ]';
      document.getElementById('json-string').textContent = jsonString;
      const fruitArray = JSON.parse(jsonString);
      document.getElementById('array-length').textContent = fruitArray.length;
      document.getElementById('first-element').textContent = fruitArray[0];
      document.getElementById('last-element').textContent = fruitArray[fruitArray.length - 1];
      console.log("Parsed Array:", fruitArray);
   </script>

</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:27 DATE: 04.9.2025	**Create a JSON file using your own data and access the JSON values by array indexing and looping**

AIM:

To write a program to create a JSON file using your own data and access the JSON values by array indexing and looping

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
  <title>JSON Array Access Demo</title>
  <style>
    body { font-family: Arial, sans-serif; }
    code { background-color: #eee; padding: 2px 4px; border-radius: 3px; }
    .output-box { border: 1px solid #ccc; padding: 10px; margin-top: 10px; }
  </style>
</head>
<body>
  <h1>Accessing JSON Array Elements</h1>
  <p>The JSON structure contains a list of <code>products</code>.</p>

  <div id="indexed-output">
    <h2>1. Access by Indexing</h2>
    </div>

  <div id="looped-output">
    <h2>2. Access by Looping</h2>
    <ul id="product-list">
      </ul>
  </div>

  <script>
    const jsonString = `
    {
      "storeName": "Tech Hub",
      "location": "Online",
      "products": [
        { "id": 101, "name": "Laptop", "price": 1200 },
        { "id": 102, "name": "Mouse", "price": 25 },
        { "id": 103, "name": "Monitor", "price": 300 }
      ]
    }`;
    const data = JSON.parse(jsonString);
    const productsArray = data.products;
    const secondProductName = productsArray[1].name;
    const secondProductPrice = productsArray[1].price;
```

```
    document.getElementById('indexed-output').innerHTML += `
        <p>Second product (Index 1) is: <strong>${secondProductName}</strong>.</p>
        <p>Its price is: <strong>$${secondProductPrice}</strong>.</p>
    `;
    const listContainer = document.getElementById('product-list');
    for (let i = 0; i < productsArray.length; i++) {
        const product = productsArray[i];
        const listItem = document.createElement('li');
        listItem.textContent = `ID: ${product.id} - ${product.name} costs $${product.price}`;
        listContainer.appendChild(listItem);
    }
</script>
</body>
</html>
```

OUTPUT :

Accessing JSON Array Elements

The JSON structure contains a list of `products`.

1. Access by Indexing

Second product (Index 1) is: **Mouse**.

Its price is: **$25**.

2. Access by Looping

- ID: 101 - Laptop costs $1200
- ID: 102 - Mouse costs $25
- ID: 103 - Monitor costs $300

RESULT :

Thus the above program was executed successfully.

EX.NO:28 DATE: 05.9.2025	**Illustrate the usage of date and function in JSON**

AIM:

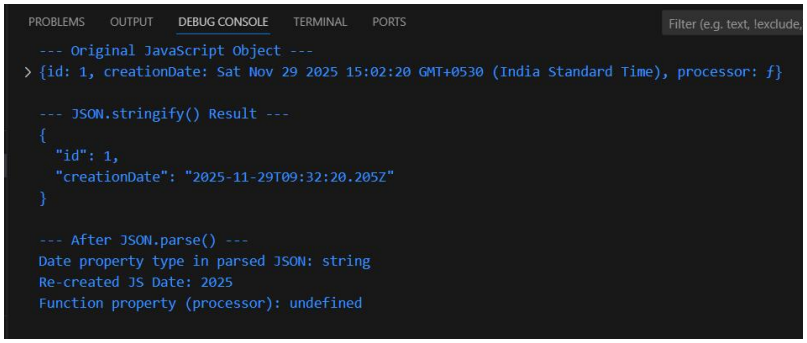
To write a program to illustrate the usage of date and function in JSON

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
  <title>JSON Date/Function Usage</title>
</head>
<body>
  <h1>JSON Stringification Demo</h1>
  <p>Check the browser console (F12) for the results.</p>
  <script>
    const data = {
      id: 1,
      creationDate: new Date(),
      processor: function(x) { return x * 2; }
    };
    const jsonString = JSON.stringify(data, null, 2);
    console.log("--- Original JavaScript Object ---");
    console.log(data);
    console.log("\n--- JSON.stringify() Result ---");
    console.log(jsonString);
    const parsedData = JSON.parse(jsonString);
    const dateFromJSON = new Date(parsedData.creationDate);
    console.log("\n--- After JSON.parse() ---");
    console.log("Date property type in parsed JSON:", typeof parsedData.creationDate);
    console.log("Re-created JS Date:", dateFromJSON.getFullYear());
    console.log("Function property (processor):", parsedData.processor);
  </script>
</body>
</html>
```

OUTPUT :





RESULT :

Thus the above program was executed successfully.

EX.NO:29 DATE: 05.9.2025	**Illustrate the usage of JSON.stringify() on array, date and function**

AIM:

To write a program to illustrate the usage of JSON.stringify() on array,date and function

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
  <title>JSON.stringify() Usage Demo</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; }
    pre { background-color: #f4f4f4; padding: 15px; border: 1px solid #ddd; overflow-x:
    auto; }
    .result-box { margin-bottom: 20px; padding: 10px; border-left: 3px solid #007bff; }
    .label { font-weight: bold; color: #333; }
  </style>
</head>
<body>
  <h1>Using JSON.stringify()</h1>
  <div id="output">
    </div>
  <script>
    const myArray = [10, "hello", true, null];
    const stringifiedArray = JSON.stringify(myArray);
    const myDate = new Date();
    const stringifiedDate = JSON.stringify(myDate);
    const myObjectWithFunction = {
      id: 1,
      name: "Test",
      calculate: function(a, b) { return a + b; }
    };
    const stringifiedObject = JSON.stringify(myObjectWithFunction);
    function displayResult(title, original, stringified) {
      const outputDiv = document.getElementById('output');
      const originalDisplay = typeof original === 'object' && original !== null &&
      original.constructor !== Array ?
                    JSON.stringify(original, null, 2) :
                    original;
      outputDiv.innerHTML += `
        <div class="result-box">
          <h3>${title}</h3>
          <p class="label">Original Data Type:</p>
          <pre><code>${originalDisplay}</code></pre>
          <p class="label">Stringified JSON Output:</p>
          <pre><code>${stringified}</code></pre>
```

```
                <p class="label">Conclusion:</p>
                <p>
                  ${
                    title.includes("Function")
                    ? "The function was **removed** from the final JSON string."
                    : "The data was successfully converted into a JSON **string
                        representation**."
                  }
                </p>
              </div>
            `;
        }
        displayResult("1. Array (`[ ]`)", myArray, stringifiedArray);
        displayResult("2. Date Object (`new Date()`)", myDate, stringifiedDate);
        displayResult("3. Object with Function", myObjectWithFunction, stringifiedObject);

    </script>
</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX.NO:30 DATE: 06.9.2025	**Program to send HTML form data to JSON file**

AIM:

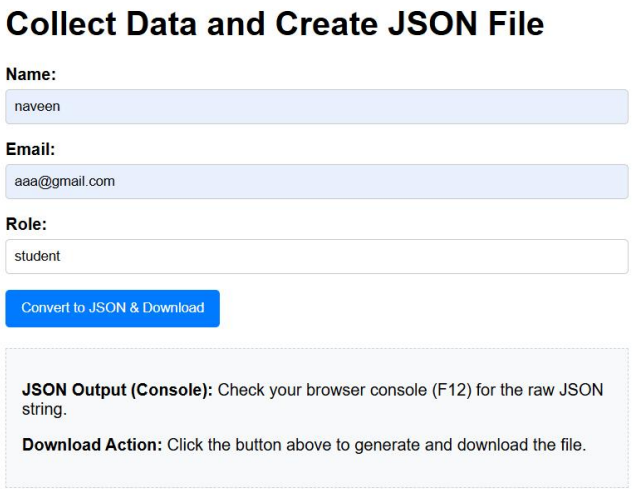
To write a program to send HTML form data to JSON file

PROGRAM:

```
<!DOCTYPE html>
<html>
<head>
  <title>Form Data to JSON Download</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; max-width: 600px; margin: auto; }
    label { display: block; margin-bottom: 5px; font-weight: bold; }
    input[type="text"], input[type="email"] { width: 100%; padding: 8px; margin-bottom: 15px; border: 1px solid #ccc; border-radius: 4px; box-sizing: border-box; }
    button { padding: 10px 15px; background-color: #007bff; color: white; border: none; border-radius: 4px; cursor: pointer; }
    button:hover { background-color: #0056b3; }
    .output-box { margin-top: 20px; padding: 15px; background-color: #f8f9fa; border: 1px dashed #ced4da; }
  </style>
</head>
<body>
  <h1>Collect Data and Create JSON File</h1>
  <form id="dataForm">
    <label for="userName">Name:</label>
    <input type="text" id="userName" name="name" required>
    <label for="userEmail">Email:</label>
    <input type="email" id="userEmail" name="email" required>
    <label for="userRole">Role:</label>
    <input type="text" id="userRole" name="role" required>
    <button type="submit">Convert to JSON & Download</button>
  </form>
  <div class="output-box">
    <p><strong>JSON Output (Console):</strong> Check your browser console (F12) for the raw JSON string.</p>
    <p><strong>Download Action:</strong> Click the button above to generate and download the file.</p>
  </div>
  <script>
    document.getElementById('dataForm').addEventListener('submit', function(event) {
      event.preventDefault();
      const formData = new FormData(event.target);
      const dataObject = {};
      formData.forEach((value, key) => {
        dataObject[key] = value;
      });
```

```
            dataObject.submissionDate = new Date().toISOString();
            const jsonString = JSON.stringify(dataObject, null, 2);
            console.log("JSON Data to be Saved:", jsonString);
            const blob = new Blob([jsonString], { type: 'application/json' });
            const url = URL.createObjectURL(blob);
            const a = document.createElement('a');
            a.href = url;
            a.download = 'form_data_' + Date.now() + '.json';
            document.body.appendChild(a);
            a.click();
            document.body.removeChild(a);
            URL.revokeObjectURL(url);
        });
    </script>
</body>
</html>
```

OUTPUT :



RESULT :

Thus the above program was executed successfully.

EX. NO: 31 DATE: 07.09.2025	DISPLAY THE DATA FROM JSON FILE TO HTML FORMS

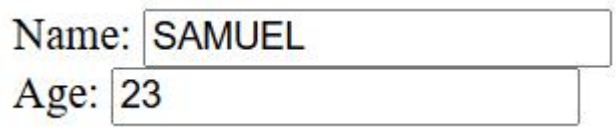
AIM:

To display JSON data in an HTML form.

CODE:

```
<!DOCTYPE html>
<html>
<head><title>Display JSON to Form</title></head>
<body>
<form>
  Name: <input type="text" id="name"><br>
  Age: <input type="text" id="age"><br>
</form>
<script>
  fetch("formdata.json")
    .then(res => res.json())
    .then(data => {
      document.getElementById("name").value = data.name;
      document.getElementById("age").value = data.age;
    });
</script>
</body>
</html>
```

OUTPUT:



RESULT:

Thus, the above program was executed successfully.

EX. NO: 32 DATE: 07.09.2025	PROGRAM TO STORE HTML FORM DATA (MORE THAN ONE RECORD) AS A SEPARATE JSON FILE.

AIM:

To store multiple records as individual JSON files.

CODE:

//index.html

```
<!DOCTYPE html>
<html>
<head><title>Multiple Record JSON</title></head>
<body>
<form method="post" action="save_multi.php">
  Name: <input type="text" name="name"><br>
  Age: <input type="text" name="age"><br>
  <input type="submit" value="Save Record">
</form>
</body>
</html>
```

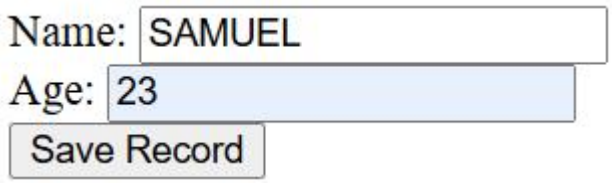
//save_multi.php

```
<?php
$name = $_POST['name'];
$age = $_POST['age'];
$data = array("name"=>$name, "age"=>$age);

$fileName = "record_" . time() . ".json";
file_put_contents($fileName, json_encode($data));

echo "Record saved as $fileName";
?>
```

OUTPUT:



Record saved as record_1764424134.json

RESULT:

Thus, the above program was executed successfully.

EX. NO: 33 DATE: 07.09.2025	IMPLEMENT THE TYPES OF EXPRESSIONS IN ANGULARJS

AIM:

To implement and demonstrate the types of expressions in AngularJS.

CODE:

```
<!DOCTYPE html>
<html ng-app>
<head><title>AngularJS Expressions</title></head>
<body>
<h2>AngularJS Expressions</h2>
<div>
  <p>Numeric Expression: {{5 + 3}}</p>
  <p>String Expression: {{"Hello" + " World"}}</p>
  <p>Boolean Expression: {{5 > 2}}</p>
  <p>Object Expression: {{ {'name':'Sam', 'age':20}.name }}</p>
  <p>Array Expression: {{ [10, 20, 30][1] }}</p>
</div>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</body>
</html>
```

OUTPUT:

AngularJS Expressions

Numeric Expression: 8

String Expression: Hello World

Boolean Expression: true

Object Expression: Sam

Array Expression: 20

RESULT:

Thus, the above program was executed successfully.

EX. NO: 34 DATE: 08.09.2025	**WRITE A CODE TO IMPLEMENT THE FOLLOWING CLASSES OF NG-MODEL DIRECTIVE:** **1. ng-empty 2. ng-not-empty 3. ng-touched 4. ng-untouched 5. ng-valid 6. ng-invalid 7. ng-dirty 8. ng-pending 9. ng-pristine**

AIM:

To demonstrate and observe the working of AngularJS form field state classes: ng-empty, ng-not-empty, ng-touched, ng-untouched, ng-valid, ng-invalid, ng-dirty, ng-pending, ng-pristine.

CODE:

```
<!DOCTYPE html>
<html lang="en" ng-app>
<head>
 <meta charset="UTF-8">
 <title>ng-model Directive Classes</title>
 <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
 <style>
  body { font-family: Arial; margin: 30px; }

  input {
   display: block;
   margin: 10px 10px;
   padding: 8px;
   width: 250px;
   border: 2px solid gray;
   border-radius: 5px;
  }

  /* Visual cues for each class */
  .ng-empty { background-color: #f8d7da; }
  .ng-not-empty { background-color: #d4edda; }
  .ng-touched { border-color: blue; }
  .ng-untouched { border-color: orange; }
```

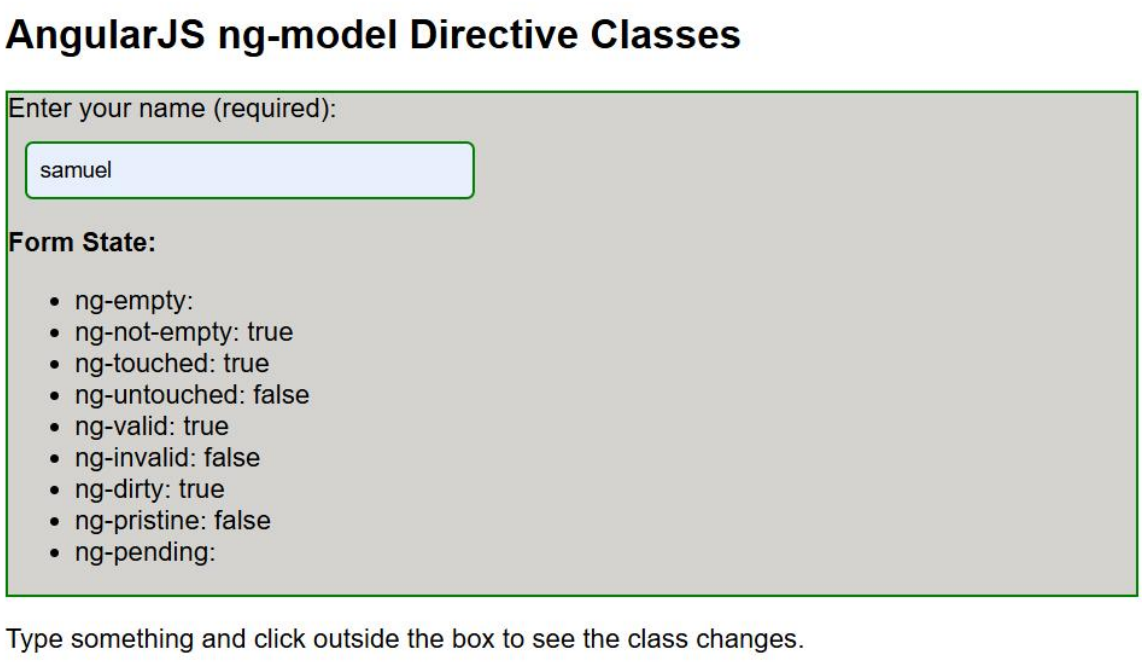
```
    .ng-valid { border: 2px solid green; }
    .ng-invalid { border: 2px solid red; }
    .ng-dirty { background-color: #d4d3d0; }
    .ng-pristine { background-color: #e2e3e5; }
  </style>
</head>
<body>
  <h2>AngularJS ng-model Directive Classes</h2>

  <form name="myForm">
    <label>Enter your name (required):</label>
    <input type="text" name="username" ng-model="user.name" required>

    <p><b>Form State:</b></p>
    <ul>
      <li>ng-empty: {{ myForm.username.$error.required }}</li>
      <li>ng-not-empty: {{ !myForm.username.$error.required }}</li>
      <li>ng-touched: {{ myForm.username.$touched }}</li>
      <li>ng-untouched: {{ myForm.username.$untouched }}</li>
      <li>ng-valid: {{ myForm.username.$valid }}</li>
      <li>ng-invalid: {{ myForm.username.$invalid }}</li>
      <li>ng-dirty: {{ myForm.username.$dirty }}</li>
      <li>ng-pristine: {{ myForm.username.$pristine }}</li>
      <li>ng-pending: {{ myForm.username.$pending }}</li>
    </ul>
  </form>

  <p>Type something and click outside the box to see the class changes. </p>
</body>
</html>
```

OUTPUT:



RESULT:

Thus, the above program was executed successfully.

EX. NO: 35 DATE: 08.09.2025	IMPLEMENT THE TYPES OF DATA BINDING IN ANGULARJS

AIM:

To implement the different types of data binding in AngularJS.

CODE:

```
<!DOCTYPE html>
<html ng-app="bindApp">
<head>
  <title>Types of Data Binding</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="bindCtrl">
  <h2>Types of Data Binding in AngularJS</h2>
  <!-- One-way binding -->
  <h3>1. One-way Binding ({{ }})</h3>
  <p>Hello, {{ name }}</p>

  <!-- Two-way binding -->
  <h3>2. Two-way Binding (ng-model)</h3>
  <input type="text" ng-model="message">
  <p>You typed: {{ message }}</p>
  <script>
    var app = angular.module("bindApp", []);
    app.controller("bindCtrl", function($scope) {
      $scope.name = "Sam";
      $scope.message = "Type something...";
    })
  </script>
</body>
</html>
```

OUTPUT:

Types of Data Binding in AngularJS

1. One-way Binding ()

Hello, Sam

2. Two-way Binding (ng-model)

Sam

You typed: Sam

RESULT:

Thus, the above program was executed successfully.

EX. NO: 36 DATE: 08.09.2025	IMPLEMENT ANGULARJS MODULES AND CONTROLLER DIRECTIVES WITH NECESSARY EXAMPLES.

AIM:

To implement AngularJS modules and controllers.

CODE:

```
<!DOCTYPE html>
<html ng-app="studentApp">
<head><title>Modules and Controllers</title></head>
<body ng-controller="studentCtrl">
<h2>AngularJS Module and Controller</h2>
<p>Name: {{student.name}}</p>
<p>Course: {{student.course}}</p>

<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
<script>
  angular.module("studentApp", [])
  .controller("studentCtrl", function($scope) {
    $scope.student = {name:"Sam", course:"Computer Science"};
  });
</script>
</body>
</html>
```

OUTPUT:

AngularJS Module and Controller

Name: Sam

Course: Computer Science

RESULT:

Thus, the above program was executed successfully.

EX. NO: 37 DATE: 09.09.2025	IMPLEMENT ANY TEN ANGULARJS FILTERS

AIM:

To implement ten filters in AngularJS.

CODE:

```
<!DOCTYPE html>
<html ng-app="filterApp" ng-controller="filterCtrl">
<head><title>AngularJS Filters</title></head>
<body>
<h2>AngularJS Filters Example</h2>
<p>1. Uppercase: {{name | uppercase}}</p>
<p>2. Lowercase: {{name | lowercase}}</p>
<p>3. Currency: {{price | currency:"₹"}}</p>
<p>4. Number: {{num | number:2}}</p>
<p>5. LimitTo: {{names | limitTo:3}}</p>
<p>6. OrderBy: {{students | orderBy:'marks'}}</p>
<p>7. Filter: {{students | filter:'A'}}</p>
<p>8. JSON: {{students | json}}</p>
<p>9. Date: {{today | date:'medium'}}</p>
<p>10. Custom Filter: {{text | reverse}}</p>

<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
<script>
  angular.module("filterApp", [])
  .controller("filterCtrl", function($scope) {
    $scope.name = "Sam";
    $scope.price = 99.99;
    $scope.num = 1234.567;
    $scope.names = ["Asha", "Ravi", "Sam", "Kiran"];
    $scope.students = [{name:"A", marks:70},{name:"B",marks:90}];
```

```
    $scope.today = new Date();
    $scope.text = "Hello";
  })
  .filter('reverse', function() {
    return function(input) {
      return input.split("").reverse().join("");
    };
  });
</script>
</body>
</html>
```

OUTPUT:

AngularJS Filters Example

1. Uppercase: SAM
2. Lowercase: sam
3. Currency: ₹99.99
4. Number: 1,234.57
5. LimitTo: ["Asha","Ravi","Sam"]
6. OrderBy: [{"name":"A","marks":70},{"name":"B","marks":90}]
7. Filter: [{"name":"A","marks":70}]
8. JSON: [{ "name": "A", "marks": 70 }, { "name": "B", "marks": 90 }]
9. Date: Nov 29, 2025 6:32:40 PM
10. Custom Filter: olleH

RESULT:

Thus, the above program was executed successfully.

EX. NO: 38 DATE: 09.09.2025	ILLUSTRATE THE FOLLOWING SERVICES OF $HTTP IN ANGULARJS 1. $http.get() 2. $http.head() 3. $http.post() 4. $http.put() 5. $http.delete() 6. $http.jsonp() 7. $http.patch()

AIM:

To illustrate the use of $http services like GET, HEAD, POST, PUT, DELETE, JSONP, and PATCH in AngularJS.

CODE:

```
<!DOCTYPE html>
<html lang="en" ng-app="httpApp">
<head>
  <meta charset="UTF-8">
  <title>$http Services Example</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <style>
    body { font-family: Arial; margin: 30px; }
    button { margin: 5px; padding: 8px 12px; cursor: pointer; }
    pre { background: #f4f4f4; padding: 10px; border-radius: 5px; }
  </style>
</head>
<body ng-controller="httpCtrl">

  <h2>$http Services in AngularJS</h2>
  <p>Click the buttons below to simulate different $http methods.</p>

  <button ng-click="getData()">$http.get()</button>
  <button ng-click="headData()">$http.head()</button>
  <button ng-click="postData()">$http.post()</button>
  <button ng-click="putData()">$http.put()</button>
  <button ng-click="deleteData()">$http.delete()</button>
  <button ng-click="jsonpData()">$http.jsonp()</button>
```

```
<button ng-click="patchData()">$http.patch()</button>

<h3>Response:</h3>
<pre>{{response | json}}</pre>

<script>
// AngularJS App and Controller
angular.module("httpApp", [])
.controller("httpCtrl", function($scope, $http) {

  const url = "https://jsonplaceholder.typicode.com/posts/1"; // free test API

  // GET Method
  $scope.getData = function() {
    $http.get(url).then(function(res) {
      $scope.response = res.data;
    });
  };

  // HEAD Method
  $scope.headData = function() {
    $http.head(url).then(function(res) {
      $scope.response = res.headers();
    });
  };

  // POST Method
  $scope.postData = function() {
    $http.post("https://jsonplaceholder.typicode.com/posts", {
      title: "New Post", body: "Sample content", userId: 1
```

```
  }).then(function(res) {
    $scope.response = res.data;
  });
};

// PUT Method
$scope.putData = function() {
  $http.put(url, { title: "Updated Post", body: "Edited content", userId: 1 })
  .then(function(res) {
    $scope.response = res.data;
  });
};

// DELETE Method
$scope.deleteData = function() {
  $http.delete(url).then(function(res) {
    $scope.response = { message: "Data deleted successfully!" };
  });
};

// JSONP Method
$scope.jsonpData = function() {
  $http.jsonp("https://api.github.com/users/angular?callback=JSON_CALLBACK")
  .then(function(res) {
    $scope.response = res.data;
  });
};

// PATCH Method
$scope.patchData = function() {
```

```
    $http.patch(url, { title: "Patched Title" }).then(function(res) {
      $scope.response = res.data;
    });
  };
});
</script>
</body>
</html>
```

OUTPUT:



RESULT:

Thus, the above program was executed successfully

EX. NO: 39 DATE: 09.09.2025	DISPLAY TABLE USING ANGULARJS NG-REPEAT DIRECTIVE (WITHOUT PHP AND WITH PHP). USE NECESSARY CSS.

AIM:

To write a program for DISPLAY TABLE USING ANGULARJS NG-REPEAT DIRECTIVE (WITHOUT PHP AND WITH PHP). USE NECESSARY CSS.

CODE:

A. Without PHP (static data)

```
<!-- table_static.html -->
<!DOCTYPE html>
<html ng-app="tableApp">
<head>
  <meta charset="utf-8" />
  <title>ng-repeat — Static Table</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <style>
    body { font-family: Arial, sans-serif; margin: 30px; }
    table { border-collapse: collapse; width: 80%; max-width: 800px; }
    th, td { padding: 10px 12px; border: 1px solid #ccc; text-align: left; }
    thead { background: #2c7be5; color: white; }
    tbody tr:nth-child(even) { background: #f6f8fb; }
    caption { font-size: 1.2em; margin-bottom: 8px; font-weight: bold; }
  </style>
</head>
<body ng-controller="TableCtrl">

  <table>
    <caption>Student List (Static)</caption>
    <thead>
      <tr><th>#</th><th>Name</th><th>Age</th><th>Course</th></tr>
    </thead>
```

```
    <tbody>
      <tr ng-repeat="s in students track by $index">
        <td>{{ $index + 1 }}</td>
        <td>{{ s.name }}</td>
        <td>{{ s.age }}</td>
        <td>{{ s.course }}</td>
      </tr>
    </tbody>
  </table>

  <script>
    angular.module('tableApp', [])
      .controller('TableCtrl', function($scope) {
        $scope.students = [
          { name: 'Sam', age: 20, course: 'CSE' },
          { name: 'Asha', age: 21, course: 'ECE' },
          { name: 'Ravi', age: 22, course: 'ME' },
          { name: 'Kiran', age: 19, course: 'IT' }
        ];
      });
  </script>
</body>
</html>
```

B. With PHP (data served from server)

```
// students.php
<?php
header('Content-Type: application/json');

// Example data array
$students = [
  ["name" => "Sam",   "age" => 20, "course" => "CSE"],
```

```
  ["name" => "Asha",  "age" => 21, "course" => "ECE"],
  ["name" => "Ravi",  "age" => 22, "course" => "ME"],
  ["name" => "Kiran", "age" => 19, "course" => "IT"]
];

// Output JSON (pretty print optional)
echo json_encode($students, JSON_PRETTY_PRINT);
```

<!-- table_php.html -->

```
<!DOCTYPE html>
<html ng-app="tableAppServer">
<head>
  <meta charset="utf-8" />
  <title>ng-repeat — Table from PHP (JSON)</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <style>
    body { font-family: Arial, sans-serif; margin: 30px; }
    table { border-collapse: collapse; width: 90%; max-width: 900px; }
    th, td { padding: 10px 12px; border: 1px solid #ddd; text-align: left; }
    thead { background: #28a745; color: #fff; }
    tbody tr:hover { background: #eef9f0; }
    caption { font-weight: bold; margin-bottom: 8px; }
    .loading { color: #666; margin-top: 12px; }
    .error { color: red; margin-top: 12px; }
  </style>
</head>
<body ng-controller="ServerTableCtrl">

  <table ng-if="students.length">
    <caption>Student List (From PHP)</caption>
    <thead>
```

```
    <tr><th>#</th><th>Name</th><th>Age</th><th>Course</th></tr>
  </thead>
  <tbody>
    <tr ng-repeat="s in students track by $index">
      <td>{{ $index + 1 }}</td>
      <td>{{ s.name }}</td>
      <td>{{ s.age }}</td>
      <td>{{ s.course }}</td>
    </tr>
  </tbody>
</table>

<div class="loading" ng-if="loading">Loading data from server...</div>
<div class="error" ng-if="error">{{ error }}</div>

<script>
  angular.module('tableAppServer', [])
    .controller('ServerTableCtrl', function($scope, $http) {
      $scope.students = [];
      $scope.loading = true;
      $scope.error = null;

      // Adjust path if PHP file is in a different folder
      $http.get('students.php')
        .then(function(response){
          $scope.students = response.data;
          $scope.loading = false;
        })
        .catch(function(err){
          $scope.error = 'Failed to load data. Check students.php and server.';
          $scope.loading = false;
```

```
        console.error(err);
      });
  });
 </script>
</body>
</html>
```

OUTPUT:

A.

Student List (Static)

#	Name	Age	Course
1	Sam	20	CSE
2	Asha	21	ECE
3	Ravi	22	ME
4	Kiran	19	IT

B.

Student List (From PHP)

#	Name	Age	Course
1	Sam	20	CSE
2	Asha	21	ECE
3	Ravi	22	ME
4	Kiran	19	IT

RESULT:

Thus, the above program was executed successfully.

EX. NO: 40 DATE: 09.09.2025	ILLUSTRATE ABOUT BINDING APPLICATION DATA TO THE ATTRIBUTES OF HTML DOM ELEMENTS.

AIM:

To bind application data to HTML element attributes.

CODE:

```
<!DOCTYPE html>
<html ng-app="bindApp" ng-controller="bindCtrl">
<head><title>Attribute Binding</title></head>
<body>
<h2>Binding Data to Attributes</h2>
<img ng-src="{{image}}" width="150">
<br>
<a ng-href="{{link}}">Go to Google</a>

<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
<script>
angular.module("bindApp", [])
.controller("bindCtrl", function($scope) {
  $scope.image =
"https://th.bing.com/th/id/R.fa21749d39d8ff8622c2c237ceb5f748?rik=f3h%2b%2fFY2Mm6
WVg&riu=http%3a%2f%2fwww.themarysue.com%2fwp-
content%2fuploads%2f2015%2f04%2fspider-
man.jpg&ehk=rp3sX0qoTecfB0bn3ODwXhKXeCl6JKb%2brMxg59H6n4E%3d&risl=&pid
=ImgRaw&r=0";
  $scope.link = "https://www.google.com";
});
</script>
</body>
</html>
```

OUTPUT:

Binding Data to Attributes



Go to Google

RESULT:

Thus, the above program was executed successfully

EX.NO:41 DATE: 24/09/2025	ANGULARJS EVENT DIRECTIVES

AIM:

To Implement the following AngularJS even directives ng-blur ng-change ng-click ng-dblclick ng-focus ng-keydown ng-keyup ng-keypress ng-mousedown ng-mouseenter ng-mouseleave ng-mousemove ng-mouseover ng-mouseup

SOURCE CODE:

```
<!DOCTYPE html>
<html ng-app="eventApp">
<head>
 <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>

 <style>
  body{ font-family:Arial; margin:25px; background:#fafafa;}
  .grid{display:grid;grid-template-columns: 1fr 1fr; gap:20px;}
  .box{border:1px solid #ccc;padding:15px;border-radius:6px;background:white;}
  .event-text{color:#1a73e8;font-weight:bold;margin-top:6px;min-height:20px;}
  .mouse-area{height:120px;border:2px dashed #666;margin-top:8px;display:flex;justify-
content:center;align-items:center;}
  button{color: #fafafa; background-color: black;}
  h2{ margin-bottom:20px; }
  @media(max-width:800px){.grid{ grid-template-columns:1fr; } }
 </style>
</head>
<body ng-controller="evCtrl">
<center><h2>AngularJS Event </h2> </center>
<div class="grid">
 <!-- FOCUS / BLUR -->
 <div class="box">
  <h3>Focus / Blur</h3>
  <input type="text"
      placeholder="Focus / Blur"
      ng-focus="focusText='Focused'"
```

```
   ng-blur="focusText='Blurred'">
  <div class="event-text">{{focusText}}</div>
</div>

<!-- ng-change -->
<div class="box">
  <h3>ng-change</h3>
  <input type="text" ng-model="t"
         ng-change="changeText='Changed: ' + t"
         placeholder="Type something">
  <div class="event-text">{{changeText}}</div>
</div>

<!-- CLICK + DOUBLE CLICK -->
<div class="box">
  <h3>Click / Double Click</h3>
  <button ng-click="detectClick()">Click Me</button>
  <div class="event-text">{{clickText}}</div>
</div>

<!-- Keyboard -->
<div class="box">
  <h3>Keyboard Events</h3>
  <input type="text"
         placeholder="Press keys"
         ng-keydown="keyText='Key Down'"
         ng-keyup="keyText='Key Up'"
         ng-keypress="keyText='Key Press'">
  <div class="event-text">{{keyText}}</div>
</div>

<!-- Mouse -->
<div class="box">
  <h3>Mouse Events</h3>
  <div class="mouse-area"
       ng-mousedown="mouseText='Mouse Down'"
```

```
        ng-mouseup="mouseText='Mouse Up'"
        ng-mouseenter="mouseText='Mouse Enter'"
        ng-mouseleave="mouseText='Mouse Leave'"
        ng-mouseover="mouseText='Mouse Over'"
        ng-mousemove="mouseMove()">
      Move / Click Here
    </div>
    <div class="event-text">{{mouseText}}</div>
  </div>
</div>

<script>
angular.module("eventApp",[])
.controller("evCtrl", function($scope, $timeout){
  /* -------- Single/Double Click Logic -------- */
  let clickCount = 0;

  $scope.detectClick = function(){
    clickCount++;

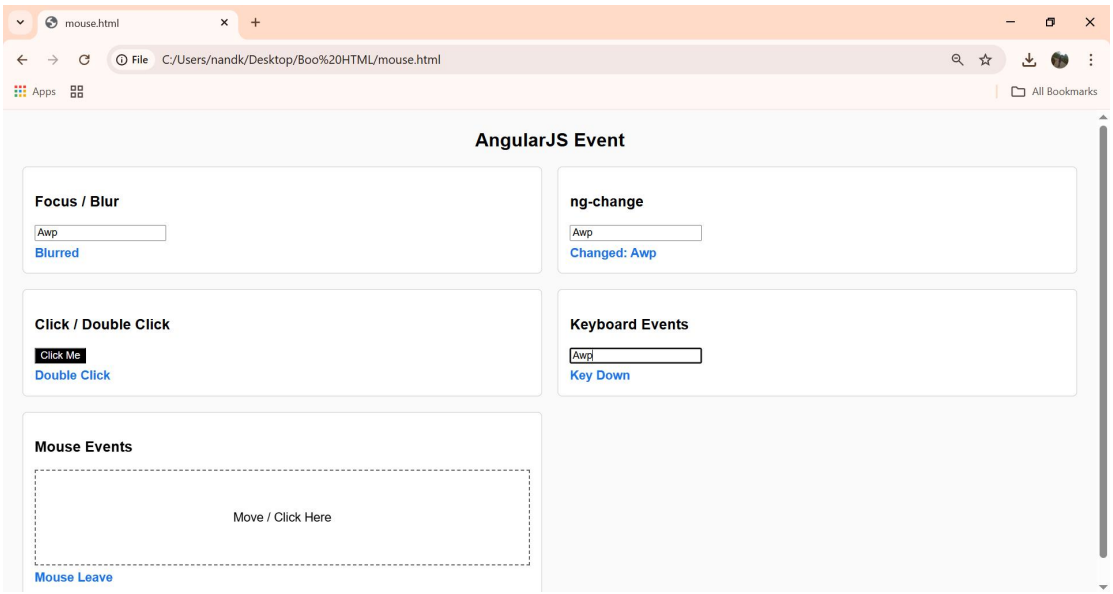
    $timeout(function(){
      if(clickCount === 1){
        $scope.clickText = "Single Click";
      } else if(clickCount === 2){
        $scope.clickText = "Double Click";
      }
      clickCount = 0;
    }, 250);
  };

  /* -------- Mouse Move Throttle -------- */
  let last = 0;
  $scope.mouseText = "—";

  $scope.mouseMove = function(){
    const now = Date.now();
```

```
    if(now - last > 200){
    $scope.mouseText = "Mouse Move";
    last = now;
   }
  };
});
</script>
</body>
</html>
```

OUTPUT:



RESULT:

Thus the program has been executed successfully.

EX.NO:42 DATE: 24/09/2025	FORM VALIDATION USING ANGULARJS

AIM:

To Design any user registration form (should include all input types) in AngularJS and validate all the fields in the form.

SOURCE CODE:

```
<!DOCTYPE html>
<html ng-app="regApp">
<head>
 <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>

 <style>
  body{ font-family:Arial; margin:30px; }
  .box{ width:350px; margin:auto; padding:20px; border:1px solid #ccc; border-radius:8px; }
  label{ font-weight:bold; margin-top:12px; display:block; }
 input, select{ width:100%; padding:8px; margin-top:6px; border:1px solid #bbb; border-
      radius:5px; }
  .error{ color:#e63946; font-size:13px; margin-top:4px; }
  button{
   margin-top:20px; width:100%; padding:10px;
   background:#1a73e8; color:white; border:none; border-radius:6px;
   font-size:16px; cursor:pointer;
  }
  button:disabled{ opacity:0.5; cursor:not-allowed; }
 </style>
</head>

<body ng-controller="formCtrl">

<div class="box">
 <h2>Registration Form</h2>

 <form name="regForm" ng-submit="submit(regForm)" novalidate>
```

```
<!-- NAME -->
<label>Name *</label>
<input type="text" name="name" ng-model="user.name" required ng-pattern="/^[A-Za-z ]+$/">
<div class="error" ng-if="regForm.name.$touched && regForm.name.$invalid">
  Enter a valid name.
</div>

<!-- PHONE -->
<label>Phone Number *</label>
<input type="tel" name="phone" ng-model="user.phone" required ng-pattern="/^[0-9]{10}$/">
<div class="error" ng-if="regForm.phone.$touched && regForm.phone.$invalid">
  Enter 10-digit phone number.
</div>

<!-- EMAIL -->
<label>Email *</label>
<input type="email" name="email" ng-model="user.email" required>
<div class="error" ng-if="regForm.email.$touched && regForm.email.$invalid">
  Enter a valid email.
</div>

<!-- GENDER -->
<label>Gender *</label>
<select name="gender" ng-model="user.gender" required>
  <option value="">-- Select Gender --</option>
  <option value="Male">Male</option>
  <option value="Female">Female</option>
  <option value="Other">Other</option>
</select>
<div class="error" ng-if="regForm.gender.$touched && regForm.gender.$error.required">
  Select gender.
</div>

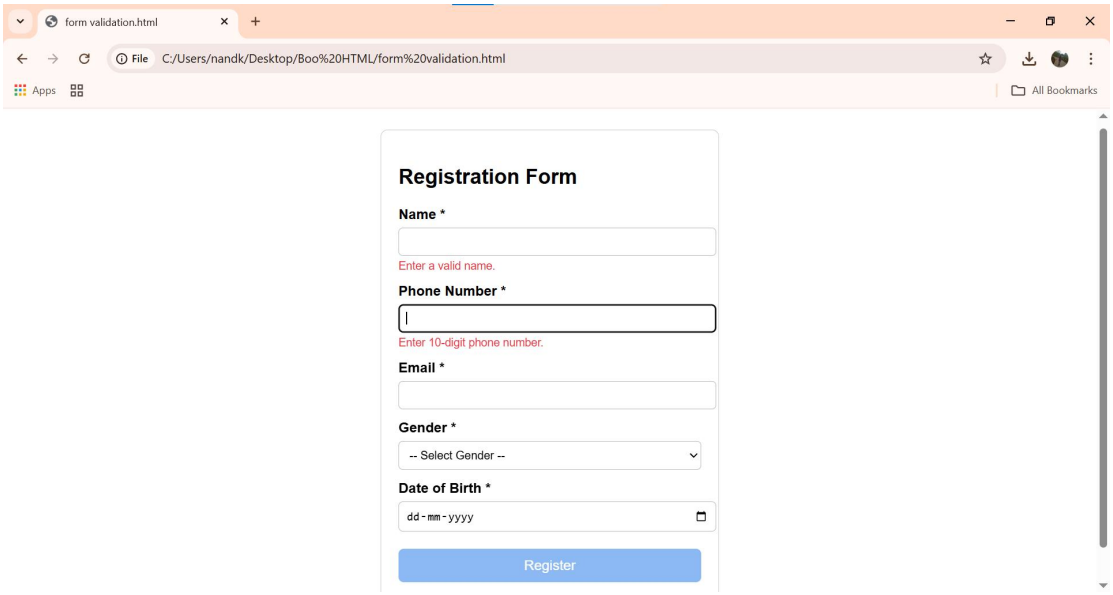
<!-- DOB -->
<label>Date of Birth *</label>
<input type="date" name="dob" ng-model="user.dob" required>
```

```
<div class="error" ng-if="regForm.dob.$touched && regForm.dob.$error.required">
      Select date of birth.
    </div>

    <!-- SUBMIT -->
    <button type="submit" ng-disabled="regForm.$invalid">Register</button>
  </form>
</div>
<script>
angular.module("regApp",[])
.controller("formCtrl", function($scope, $window){

  $scope.user = {};
  $scope.submit = function(form){
    if(form.$valid){
      $window.alert("Registration Successful!\n\n" +
                    "Name: " + $scope.user.name + "\n" +
                    "Phone: " + $scope.user.phone + "\n" +
                    "Email: " + $scope.user.email + "\n" +
                    "Gender: " + $scope.user.gender + "\n" +
                    "DOB: " + $scope.user.dob);
    } else {
      $window.alert("Please fill all required fields correctly.");
    }
  };
});
</script>
</body>
</html>
```

OUTPUT:



RESULT:

Thus the program has been executed successfully.

EX.NO:43 DATE: 25/09/2025	IMPLEMENT THE CORE MODULE

AIM:

To Implement the usage of core module in Node.JS

SOURCE CODE:

```
const fs = require('fs');      // File System
const path = require('path');  // Path Operations
const os = require('os');      // OS Information
const http = require('http');  // HTTP Server

// 1. FILE SYSTEM (fs)
fs.writeFileSync('sample.txt', 'Hello, this is Node.js core module FS!');
console.log('✔ File created using fs module.');

const content = fs.readFileSync('sample.txt', 'utf-8');
console.log('✔ File content:', content);

// 2. PATH MODULE
const fullPath = path.join(__dirname, 'sample.txt');
console.log('✔ Full file path:', fullPath);

// 3. OS MODULE
console.log('✔ Operating System:', os.type());
console.log('✔ Free Memory:', os.freemem());
console.log('✔ User Info:', os.userInfo());

// 4. HTTP MODULE – simple server
const server = http.createServer((req, res) => {
   res.writeHead(200, { 'Content-Type': 'text/plain' });
   res.end('Hello, Node.js Core Modules working!');
});

server.listen(3000, () => {
```

```
  console.log('✔ HTTP Server running at http://localhost:3000');
});
```

OUTPUT:

```
PS C:\Users\nandk\Desktop\Boo HTML> node core.js
✓ File created using fs module.
✓ File content: Hello, this is Node.js core module FS!
✓ Full file path: C:\Users\nandk\Desktop\Boo HTML\sample.txt
✓ Operating System: Windows_NT
✓ Free Memory: 3686731776
✓ User Info: [Object: null prototype] {
  uid: -1,
  gid: -1,
  username: 'nandk',
  homedir: 'C:\\Users\\nandk',
  shell: null
}
✓ HTTP Server running at http://localhost:3000
```

RESULT:

Thus the program has been executed successfully.

EX.NO:44 DATE: 08/10/2025	IMPLEMENT LOCAL AND THIRD-PARTY MODULE

AIM:

To Implement Local module in Node.JS and Third-party Module in Node.JS.

SOURCE CODE:

math.js

```
// Add two numbers
exports.add = function(a, b) {
   return a + b;
};

// Subtract two numbers
exports.subtract = function(a, b) {
   return a - b;
};
```

app.js

```
// Local module
const math = require('./math');

// Third-party module
const _ = require('lodash');

console.log("Local Module Add:", math.add(3, 7));
console.log("Local Module Sub:", math.subtract(10, 2));

const arr = [3, 6, 9, 12];
console.log("Third-party Module Sum:", _.sum(arr));
console.log("Third-party Shuffle:", _.shuffle(arr));
```

OUTPUT:

```
PS C:\Users\nandk\Desktop\Boo HTML> node app.js
Local Module Add: 10
Local Module Sub: 8
Third-party Module Sum: 30
Third-party Shuffle: [ 9, 6, 3, 12 ]
```

RESULT:

Thus the program has been executed successfully.

EX.NO:45 DATE: 08/10/2025	METHODS OF HTTP MODULE

AIM:

To Implement any five methods of HTTP module.

SOURCE CODE:

```
const http = require("http");

  const server = http.createServer((req, res) => {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.write("Welcome to Node.js HTTP Module Demo!\n");
    res.write("This is a printed message from your server.\n");
    res.write("Path: " + req.url + "\n");
    res.write("Method: " + req.method + "\n");
    res.end("Server response complete!");
});

server.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
});
```

OUTPUT:

```
Welcome to Node.js HTTP Module Demo!
This is a printed message from your server.
Path: /
Method: GET
Server response complete!
```

RESULT:

Thus the program has been executed successfully.

EX.NO:46 DATE: 08/10/2025	METHODS IN FILE SYSTEM MODULE

AIM:

To Implement the methods in file system module.

SOURCE CODE:

```
const fs = require("fs");
// 1. Write file
fs.writeFileSync("test.txt", "Hello Node FS!");
console.log("1. File created.");
// 2. Read file
let data = fs.readFileSync("test.txt", "utf-8");
console.log("2. File content:", data);
// 3. Append file
fs.appendFileSync("test.txt", "\nMore content added.");
console.log("3. Content appended.");
// 4. Rename file
fs.renameSync("test.txt", "newTest.txt");
console.log("4. File renamed.");
// 5. Delete file
fs.unlinkSync("newTest.txt");
console.log("5. File deleted.");
```

OUTPUT:

```
PS C:\Users\nandk\Desktop\Boo HTML> node file.js
1. File created.
2. File content: Hello Node FS!
3. Content appended.
4. File renamed.
5. File deleted.
```

RESULT:

Thus the program has been executed successfully.

EX.NO:47 DATE: 15/10/2025	URL MODULE

AIM:

To implement a program to Illustrate URL module using Node Js

SOURCE CODE:

```
const url = require("url");

const address = "https://www.example.com/products?item=phone&price=20000";

const parsed = url.parse(address, true);

console.log("Protocol:", parsed.protocol);
console.log("Host:", parsed.host);
console.log("Pathname:", parsed.pathname);
console.log("Query Params:", parsed.query);
console.log("Item:", parsed.query.item);
console.log("Price:", parsed.query.price);
```

OUTPUT:

```
PS C:\Users\nandk\Desktop\Boo HTML> node server.js
Protocol: https:
Host: www.example.com
Pathname: /products
Query Params: [Object: null prototype] { item: 'phone', price: '20000' }
Item: phone
Price: 20000
```

RESULT:

Thus the program has been executed successfully.

EX.NO:48 DATE: 16/10/2025	EVENT HANDLING IN NODEJS

AIM:

To Illustrate Event handling in Node.JS.

SOURCE CODE:

```
const EventEmitter = require("events");

const myEvent = new EventEmitter();

// Event listener
myEvent.on("greet", () => {
    console.log("Hello! Event triggered.");
});

// Event with argument
myEvent.on("welcome", (name) => {
    console.log("Welcome, " + name);
});

// Trigger events
myEvent.emit("greet");
myEvent.emit("welcome", "Booma");
```

OUTPUT:

```
PS C:\Users\nandk\Desktop\Boo HTML> node event.js
Hello! Event triggered.
Welcome, Booma
```

RESULT:

Thus the program has been executed successfully.

EX.NO:49 DATE: 22/10/2025	FILE UPLOADING

AIM:

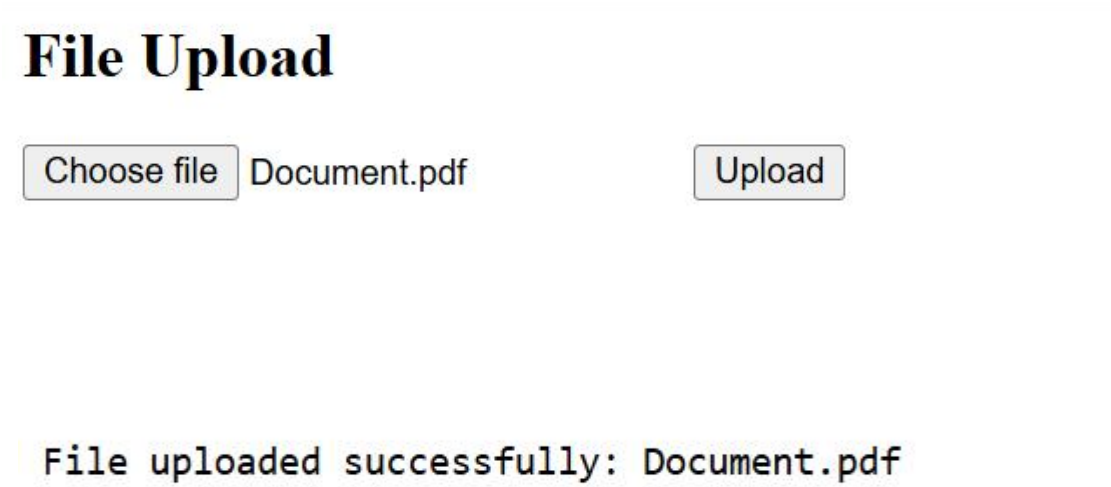
To implement a program for File uploading in Node.JS

SOURCE CODE:

```
const http = require("http");
const fs = require("fs");
if (!fs.existsSync("uploads")) {
    fs.mkdirSync("uploads");
}
const server = http.createServer((req, res) => {
    if (req.method === "GET") {
        // Simple HTML form
        res.writeHead(200, { "Content-Type": "text/html" });
        res.end(`
            <h2>File Upload </h2>
            <form method="POST" enctype="multipart/form-data">
                <input type="file" name="file" />
                <button type="submit">Upload</button>
            </form>
        `);
    }
    else if (req.method === "POST") {
        let data = Buffer.alloc(0);
        req.on("data", chunk => {
            data = Buffer.concat([data, chunk]);
        });
        req.on("end", () => {
            const boundary = req.headers['content-type'].split("boundary=")[1];
            // Convert buffer to string to find boundaries
            const parts = data.toString().split("--" + boundary);
```

```
        // The file content is in the second part
        const filePart = parts[1];
        // Extract filename
        const fileNameMatch = filePart.match(/filename="(.+?)"/);
        const fileName = fileNameMatch ? fileNameMatch[1] : "uploaded.bin";
        // Extract file data
        const fileData = filePart.split("\r\n\r\n")[1];
        const fileContent = fileData.slice(0, fileData.length - 2); // remove last \r\n
        // Save file
        fs.writeFileSync("uploads/" + fileName, fileContent, "binary");
        res.writeHead(200, { "Content-Type": "text/plain" });
        res.end("File uploaded successfully: " + fileName);
      });
  }
});
server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

OUTPUT:



RESULT:

Thus the program has been executed successfully.

EX.NO:50 DATE: 23/10/2025	SENDING EMAIL

AIM:

To implement a program for Sending Email in Node.JS

SOURCE CODE:

```
const nodemailer = require("nodemailer");
// 1. Create email transporter
let transporter = nodemailer.createTransport({ service: "gmail",
    auth: {
        user: "boomastyles17@gmail.com",
        pass: "wkwc vwse xkmj ojsw"
    }
});
// 2. Email content
let mailOptions = {
    from: "boomastyles17@gmail.com",
    to: "boomakannan17@gmail.com",
    subject: "Test Email from Node.js",
    text: "Hello! This is a test email sent using Node.js."
};
// 3. Send email
transporter.sendMail(mailOptions, (err, info) => {
    if (err) {console.log("Error:", err);  }
else {console.log("Email sent:", info.response); }
});
```

OUTPUT:

```
PS C:\Users\nandk\Desktop\Boo HTML> node emailsend.js
Email sent: 250 2.0.0 OK  1764414887_98e67ed59e1d1-3477b74c83esm7490387a91.15 - gsmtp
```

RESULT:

Thus the program has been executed successfully.